

Министерство образования Иркутской области

Государственное бюджетное профессиональное образовательное учреждение

Иркутской области

«Иркутский авиационный техникум»

(ГБПОУИО «ИАТ»)

ДП.09.02.7-1.26.221.03 ПЗ

УТВЕРЖДАЮ

Зам. Директора по УР, к.т.н.

_____ Е.А. Коробкова

ВЕБ-ПРИЛОЖЕНИЕ «НЕДВИЖИМОСТЬ»

Нормконтролер

(Н.Р. Огородникова)

(подпись, дата)

Руководитель:

(С.Е. Степанов)

(подпись, дата)

Студент:

(А.А. Березин)

(подпись, дата)

Иркутск 2026

СОДЕРЖАНИЕ

Введение.....	3
1 Предпроектное исследование	5
1.1 Описание предметной области	5
1.2 Анализ инструментальных средств для разработки.....	8
2 Техническое задание	15
3 Проектирование программного продукта	16
3.1 Архитектура программного продукта.....	16
3.2 Функциональное и структурное проектирование.....	17
3.3 Проектирование базы данных.....	20
3.4 Проектирование пользовательского интерфейса.....	42
4 Реализация программного продукта	48
4.1 Реализация базы данных	48
4.2 Реализация серверной части	49
4.3 Реализация клиентской части	51
5 Тестирование веб-приложения	54
5.1 Функциональное тестирование серверной части	54
5.2 Нагрузочное тестирование	61
6 Документирование программного продукта	66
6.1 Руководство по установке программного продукта.....	66
6.2 Руководство пользователя.....	66
6.3 Руководство администратора.....	67
6.4 Руководство модератора.....	69
6.5 Руководство агента.....	70
6.6 Руководство клиента.....	71
Заключение	73
Список используемых источников	75
Приложение А – Техническое задание	77
Приложение Б – Инфологическая модель базы данных	87
Приложение В – ER-модель базы данных	88
Приложение Г – Листинг кода viewing_payments.py	89

					ДП.09.02.7-1.26.221.03 ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата				
Разраб.		Березин А.А			ВЕБ-ПРИЛОЖЕНИЕ «НЕДВИЖИМОСТЬ»	Лит.	Лист	Листов
Провер.		Степанов С.Е					2	91
Реценз.						ГБПОУИО «ИАТ» БД-22-1		
Н. Контр.		Огородникова Н.Р						
Утверд.		Коробкова Е.А.						

Введение

В условиях активной цифровой трансформации экономики особую значимость приобретают информационные технологии, направленные на автоматизацию бизнес-процессов, повышение эффективности управления ресурсами и оптимизацию взаимодействия с клиентами. Современные компании, работающие в сфере недвижимости – агентства недвижимости, управляющие компании, отделы аренды – стремятся внедрять цифровые решения, обеспечивающие централизованное хранение данных, эффективное управление объектами и оперативную обработку клиентских запросов.

Одним из ключевых инструментов такой цифровизации является веб-приложение класса ERP/CRM, предназначенное для комплексной автоматизации деятельности в сфере недвижимости. Данная система обеспечивает централизованный учёт объектов недвижимости (коммерческой, жилой), управление взаимодействием с клиентами (арендаторами, покупателями, собственниками), фиксацию и сопровождение заявок, а также поддержку аналитики и документооборота.

Использование интегрированной ERP/CRM-системы предоставляет ряд существенных преимуществ. Централизованное хранение информации о клиентах и объектах снижает вероятность ошибок, характерных для разрозненных электронных таблиц и бумажных журналов. Автоматизация процессов поступления и обработки заявок позволяет сократить время реакции на запросы клиентов, повысить прозрачность сделок и исключить потерю важных данных. Для менеджеров создаются условия для быстрого доступа к актуальной информации об объектах и истории взаимодействия с клиентами, что способствует повышению качества обслуживания и эффективности продаж.

Веб-приложение также является эффективным инструментом контроля и аналитики. Особое внимание в проекте уделено безопасности финансовых операций: для отладки и тестирования функционала оплаты просмотров объектов

					ДП.09.02.7-1.26.221.03 ПЗ	Лист 3
Изм.	Лист	№ докум.	Подпись	Дата		

недвижимости применялась изолированная тестовая среда (песочница) ПАО «Сбербанк», что позволило гарантировать корректность обработки платёжных транзакций без риска использования реальных денежных средств. Руководство компании получает возможность анализа загрузки объектов. Сотрудники имеют доступ к персональным задачам, статусы по каждому объекту или запросу. Клиенты могут отслеживать статус своих заявок или просматривать доступные предложения.

Целью дипломного проекта является разработка веб-приложения (ERP/CRM-системы) для управления объектами недвижимости и заявками клиентов, обеспечивающего автоматизацию ключевых бизнес-процессов риелторской или управляющей компании, повышение прозрачности взаимодействия с клиентами и снижение административной нагрузки на сотрудников. Разрабатываемая система должна реализовывать функции учёта объектов недвижимости, ведения клиентской базы, управления входящими заявками, хранения истории взаимодействий, а также предоставлять удобный и интуитивно понятный интерфейс для менеджеров, агентов и администраторов.

Для осуществления обозначенной цели служат следующие задачи:

1. Анализ предметной области и требований к ERP/CRM-системе для сферы недвижимости.
2. Анализ инструментальных средств разработки.
3. Проектирование архитектуры веб-приложения.
4. Проектирование и создание базы данных для хранения информации об объектах, клиентах и заявках.
5. Разработка серверной части.
6. Разработка клиентской части.
7. Тестирование системы: функциональное, нагрузочное.
8. Устранение выявленных ошибок и оптимизация работы.
9. Подготовка пользовательской документации.
10. Разработка инструкции администратора веб-приложения.

1 Предпроектное исследование

1.1 Описание предметной области

Основным назначением разрабатываемого веб-приложения является автоматизация процессов управления объектами недвижимости и взаимодействия с клиентами, работающих на рынке аренды и продажи. Веб-приложение ориентирована на применение в агентствах недвижимости, отделах аренды или управляющих компаниях и предназначена для централизованного учёта объектов (коммерческой и жилой недвижимости), ведения клиентской базы, обработки заявок, управления задачами сотрудников и контроля сделок через веб-интерфейс.

Функционирование веб-приложения организовано следующим образом. На начальном этапе администратор осуществляет наполнение системы базовыми справочниками: добавление объектов недвижимости с их характеристиками (адрес, площадь, стоимость, статус), а также создание учётных записей для сотрудников (администратор, менеджер, агент) с назначением соответствующих ролей и прав доступа. Клиенты регистрируются самостоятельно, но без выбора роли – тип учётной записи и должность назначает администратор через отдельный интерфейс. Данные об объектах являются основой веб-приложения и обновляются по мере появления новых предложений или изменения статуса существующих.

После прохождения процедуры аутентификации пользователи получают доступ исключительно к тем функциям, которые предусмотрены их ролью. Агенты по недвижимости работают только со своими заявками и задачами: фиксируют новые запросы на покупку или аренду, подбирают подходящие объекты на основе заданных критериев, фиксируют результаты просмотров и этапы переговоров, выполняют назначенные задачи (звонок, показ, подготовка документов). Менеджеры и администраторы видят все заявки, объекты и задачи, могут назначать ответственных сотрудников и контролировать выполнение. Клиент видит только свои заявки, сделки, свои объекты, если он выставил на(продажу/аренду) свой объект, а также этапы сделки.

					ДП.09.02.7-1.26.221.03 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		

Каждый объект недвижимости в веб-приложении характеризуется набором параметров: тип операции (продажа/аренда), уникальный идентификатор, адрес, площадь, количество комнат, этаж, стоимость, статус (активен, в резерве, продан, сдан в аренду, архив) и дополнительные характеристики из справочника. Работа с клиентскими запросами организована через сущность «Заявка», которая привязывается к конкретному клиенту и интересующему его типу объектов.

Статусы заявок и объектов управляются сотрудниками и позволяют отслеживать жизненный цикл каждого предложения или запроса: от поступления нового объекта или заявки до заключения сделки. Это обеспечивает систематизацию данных и упрощает анализ текущей деятельности компании.

В результате анализа функционирования веб-приложения для управления недвижимостью был определён следующий состав информационных потоков.

Входные данные:

Сведения о пользователях веб-приложения: пароль, электронная почта, номер телефона, тип учётной записи (клиент или сотрудник), роль (администратор, менеджер, агент), обеспечивающие разграничение прав доступа и функциональных возможностей.

Данные об объектах недвижимости: тип операции (продажа/аренда), общая площадь, количество комнат, этаж, этажность дома, стоимость, статус объекта, координаты (широта/долгота) и текстовое описание.

Интеграция с сервисами подсказок адресов и детализацию данных. Для обеспечения достоверности и единообразия адресной информации при вводе объектов недвижимости и данных клиентов веб-приложение интегрируется с внешними сервисами. При вводе адреса используются подсказки от сервиса «ДаДата», который позволяет быстро и точно заполнить адресные поля. Для получения дополнительной информации об объекте недвижимости, используется сервис «ДваГИС». Обращение к внешним API выполняется только на стороне сервера, ключи доступа хранятся в файле переменных окружения и не передаются на клиентскую часть.

					ДП.09.02.7-1.26.221.03 ПЗ	Лист
						6
Изм.	Лист	№ докум.	Подпись	Дата		

Информация о клиентах: имя, фамилия, отчество, контактные данные (телефон, email), паспортные данные (для физических лиц) или реквизиты организации, адрес регистрации и фактический адрес, а также история взаимодействия для ведения клиентской базы.

Содержание заявок клиентов: данные о клиенте, тип желаемой операции (покупка/аренда), желаемый тип недвижимости, требуемые характеристики, статус заявки (открыта, в работе, завершена, отменена) и привязанные к ней показы объектов (варианты подбора).

Данные о сделках: информация о заключённых договорах, итоговая стоимость, процент и сумма комиссии агента, дата сделки, номер сделки, статус сделки, а также статус генерации PDF-договора (в очереди, обрабатывается, готов, ошибка).

Выходные данные:

Актуальная база объектов: менеджеры и агенты получают структурированную информацию по всем доступным объектам недвижимости с возможностью фильтрации по статусу, типу операции, цене, площади, количеству комнат и адресу, а также с использованием подсказок адресов от сервиса «ДаДата».

Результаты обработки заявок: список подобранных объектов под конкретный запрос клиента с возможностью подтверждения подходящего варианта, история показов и переговоров.

Отчёты и аналитика: данные по активности агентов, количеству заявок, конверсии в сделки, статистике по объектам (активные/проданные/сданные) для руководства компании, доступные в виде выгрузок в форматах CSV, XLSX, JSON.

Журнал действий пользователей и история изменений данных: фиксируются все значимые действия сотрудников (изменение статуса объекта, добавление комментария к заявке, смена статуса сделки, назначение задачи, завершение задачи).

Таким образом, разрабатываемое веб-приложение управления объектами недвижимости и заявками клиентов обеспечивает централизованную обработку,

					ДП.09.02.7-1.26.221.03 ПЗ	Лист
						7
Изм.	Лист	№ докум.	Подпись	Дата		

хранение и интеграцию данных, необходимых для автоматизации деятельности риелторской или управляющей компании. Использование сервиса подсказок адресов «ДаДата» гарантирует высокое качество и достоверность адресной информации, что является критически важным для работы с недвижимостью, а интеграция с «ДваГИС» позволяет заполнить карточки объектов координатами и визуальной информацией. Интеграция для верификации корректности функционирования модуля онлайн-оплаты в сфере недвижимости применялась изолированная тестовая среда (песочница) ПАО "Сбербанк". Веб-приложение предоставляет сотрудникам актуальную информацию об объектах и клиентах, обеспечивает контроль над процессом обработки заявок и фиксирование действий, что способствует повышению эффективности продаж и качества обслуживания.

1.2 Анализ инструментальных средств для разработки

Для создания веб-приложения «Недвижимость» был подобран набор инструментов. Выбор осуществлялся с учетом следующих критериев: скорость и простота разработки, надежность работы, производительность, наличие документации, а также соответствие техническим задачам проекта.

Draw.io: Бесплатное кроссплатформенное средство для построения диаграмм. Было использовано для создания UML-диаграмм (Use Case, Sequence Diagram) и схем взаимодействия компонентов системы для лучшего понимания архитектуры проекта.

MySQL Workbench: Визуальный инструмент для проектирования баз данных. Хотя в проекте была использована СУБД PostgreSQL, MySQL Workbench был применен на этапе проектирования для создания детальной ER-модели предметной области из-за удобного и наглядного интерфейса.

Веб-приложение состоит из двух частей: клиентской и серверной. Для реализации клиентской части используются Vue.js. Серверная часть реализована на Python с фреймворком Django и работает с базой данных PostgreSQL.

					ДП.09.02.7-1.26.221.03 ПЗ	Лист 8
Изм.	Лист	№ докум.	Подпись	Дата		

Для хранения данных требовалась надежная реляционная СУБД. Были рассмотрены три наиболее популярных варианта в таблице 1.

Таблица 1 - Сравнение СУБД

Критерий	PostgreSQL	MariaDB	SQLite
Тип	Объекционно-реляционная СУБД (ORDBMS)	Реляционная СУБД (RDBMS), форк MySQL	Встраиваемая реляционная СУБД (на основе файлов)
Производительность	Высокая, особенно на сложных запросах и больших объемах данных.	Высокая, оптимизирована для операций чтения, улучшенная производительность отдельных типов запросов по сравнению с MySQL.	Очень высокая для однопользовательского доступа, не предназначена для высоких нагрузок.
Масштабируемость	Отличная поддержка сложных структур данных, транзакций, репликации.	Унаследовала основные возможности MySQL с улучшениями в механизмах репликации и кластеризации.	Отсутствует, так как это встраиваемая БД.
Функции и стандарты	Наиболее строгое соответствие стандарту SQL, поддержка оконных функций, JSON.	Полная совместимость с MySQL с добавлением новых движков хранения и оптимизаций.	Поддерживает большинство SQL-запросов, но с ограничениями (напр., ограниченный ALTER TABLE).
Использование	Крупные и сложные проекты, где важна целостность данных и богатый функционал.	Веб-приложения среднего и крупного масштаба, высоконагруженные проекты.	Мобильные приложения, простые десктопные приложения, прототипирование.

Для данного веб-приложения была выбрана PostgreSQL. Основными причинами стали:

– Строгое соответствие стандартам SQL и поддержка сложных запросов, что важно для аналитики и отчетности.

– Надежность и целостность данных, обеспечиваемые строгой системой транзакций.

– Масштабируемость. База данных проекта потенциально может расти, и PostgreSQL предоставляет все необходимые инструменты для этого.

– Богатый набор функций, таких как работа с JSON, полнотекстовый поиск, которые могут пригодиться для будущего расширения функционала.

Бэкенд системы требует языка с богатой экосистемой для веб-разработки, простотой поддержки и надежностью в таблице 2.

Таблица 2 – Сравнение языков программирования

Критерий	Python (Django)	PHP (Laravel)	Java (Spring Boot)
Синтаксис	Простой и читаемый синтаксис, низкий порог входа.	Синтаксис прост, но может быть непоследовательным.	Строгий, вербальный синтаксис
Фреймворки	Мощные и "всеобъемлющие" фреймворки (Django), множество библиотек для любых задач.	Огромное количество готовых решений и фреймворков (Laravel, Symfony).	Огромная enterprise-экосистема (Spring).
Безопасность	Django предоставляет "из коробки" многие механизмы защиты.	Требуется большей внимательности к безопасности.	Высокий уровень безопасности
Производительность	Средняя. Вполне достаточна для данного проекта.	Ниже, чем у Python и Java.	Очень высокая.
Использование	Быстрая разработка complex-приложений, веб-сервисы.	Традиционно веб-сайты	Крупные корпоративные, высоконагруженные и распределенные системы.

В качестве основного языка программирования был выбран Python вместе с фреймворком Django. Это решение обусловлено следующими факторами:

– Скорость разработки. Фреймворк Django позволяет быстро создать каркас приложения.

– Читаемость кода и простота. Четкий синтаксис Python облегчает командную работу и будущую поддержку проекта.

– Мощный ORM. Django ORM позволяет работать с базой данных на высоком уровне абстракции.

– Безопасность. Django предоставляет встроенные и надежные механизмы защиты от распространенных уязвимостей.

Интегрированная среда разработки – ключевой инструмент, влияющий на продуктивность программиста в таблице 3

Таблица 3 - Сравнение IDE

Критерий	PyCharm (Professional)	Visual Studio Code	Sublime Text
Тип	Полнофункциональная IDE	Редактор кода с возможностями IDE	Высокопроизводительный текстовый редактор с плагинами
Поддержка Python/Django	Нативная, глубокая "из коробки".	Требуется установка расширений.	Требуется установка множества плагинов для достижения функционала IDE.
Производительность	Высокая, оптимизирована для больших проектов.	Очень высокая, быстрый запуск.	Чрезвычайно высокая, самый быстрый запуск и отклик.
Отладка	Мощный встроенный дебаггер.	Дебаггер через расширение.	Только через сторонние плагины.
Стоимость	Платная, есть бесплатный Community-вариант.	Полностью бесплатная.	Условно-бесплатная.

В качестве основной IDE был выбран PyCharm Professional. Ключевые причины:

– Исключительная поддержка Django. PyCharm предлагает лучшую на рынке интеграцию с Django: понимание структуры проекта, авто дополнение для шаблонов, моделей и представлений, удобная навигация между файлами.

– Встроенный набор инструментов. Наличие встроенного дебаггера, базы данных, терминала и инструментов для работы с Git значительно ускоряет процесс разработки и отладки.

– Умный код-ассистент. Высококачественное авто дополнение кода и анализ кода помогают избегать ошибок и писать код в соответствии со стандартами.

Обеспечение качества и надежности программного продукта является неотъемлемой частью процесса разработки. Сравнение инструментов тестирования в таблице 4.

Таблица 4 – Сравнение инструментов тестирования

Критерий	pytest	unittest (стандартный)	Django Test Case (на базе unittest)
Синтаксис и простота	Очень простой и лаконичный. Используются обычные функции и assert.	Более многословный, требует наследования от TestCase и использования специальных методов.	Аналогичен unittest, но предоставляет дополнительные assertion-методы для Django
Фикстуры	Мощная и гибкая система фикстур с широкими возможностями.	Базовые возможности setup/teardown методов.	Использует механизм setUp/tearDown, а также предоставляет готовые фикстуры (тестовый клиент, тестовая БД).
Интеграция с Django	Требуется установки дополнительных плагинов (например, pytest-django) для полноценной работы.	Интегрируется на базовом уровне.	Прямая и нативная интеграция. Создан специально для тестирования Django-приложений.
Использование	Универсальные проекты на Python, где важна простота и мощь.	Проекты, где достаточно стандартной библиотеки.	Проекты на Django. Наиболее предпочтителен для тестирования моделей, представлений, форм и шаблонов.

Для тестирования веб-приложения для управления объектами недвижимости и заявками клиентов был выбран инструмент Django Test Case. Основными причинами стали:

- нативная интеграция с Django. Это главное преимущество. Фреймворк автоматически создает тестовую базу данных, предоставляет готовый тестовый клиент для эмуляции запросов к приложению и специализированные методы для проверки ответов, что критически важно для тестирования представлений и шаблонов;

- скорость написания тестов. Для разработчика, работающего в среде Django, использование встроенного инструмента является наиболее прямым и понятным путем, не требующим изучения дополнительного синтаксиса или настройки плагинов;

– надежность. Как часть самого фреймворка Django, инструмент гарантированно совместим со всеми версиями и правильно обрабатывает все специфичные для Django аспекты.

Для создания веб-приложения было решено использовать следующие инструментальные средства:

1. Для проектирования архитектуры и баз данных использовались CASE-средства:

– Draw.io – для создания структурных схем, диаграмм потоков данных (DFD), диаграмм вариантов использования (Use Case).

– MySQL Workbench – для наглядного проектирования структуры реляционной базы данных и построения ER-диаграммы.

2. Для разработки пользовательского интерфейса (UI/UX) использовался современный онлайн-сервис:

– Draw.io – для создания детального прототипа и дизайна веб-интерфейса будущего приложения, что позволило утвердить визуальную концепцию и логику взаимодействия до начала написания кода.

3. Для реализации серверной части (Backend) использовался следующий стек технологий:

– Язык программирования Python – в качестве основного языка разработки благодаря его чистоте синтаксиса, богатой экосистеме и высокой скорости создания приложений.

– Веб-фреймворк Django – в качестве основы backend-а для быстрой разработки безопасного и поддерживаемого кода по паттерну MVC (MVT).

– СУБД PostgreSQL – в качестве надежной и мощной системы управления базами данных для хранения всей информации о помещениях, категориях, транзакциях и пользователях.

4. Для реализации клиентской части (Frontend) применялись стандартные веб-технологии:

					ДП.09.02.7-1.26.221.03 ПЗ	Лист
						13
Изм.	Лист	№ докум.	Подпись	Дата		

– HTML5, CSS3 – для разметки и стилизации веб-страниц. Глобальные стили вынесены в main.css.

– Vue 3 (Composition API) – основной фреймворк для построения интерфейса.

5. В качестве основной среды разработки (IDE) был выбран:

– PyCharm Professional – как наиболее мощная и удобная среда, обеспечивающая глубочайшую поддержку как Python, так и фреймворка Django, включая отладку, рефакторинг и работу с базой данных прямо из IDE.

6. Для тестирования веб-приложения был выбран:

– Django Test Case, потому что максимально ориентирован на предметную область и технологический стек разработки.

					ДП.09.02.7-1.26.221.03 ПЗ	Лист
						14
Изм.	Лист	№ докум.	Подпись	Дата		

2 Техническое задание

В начале разработки создавалось техническое задание, в котором указывались основные требования.

Для создания технического задания использовался стандарт ГОСТ 34.602–2020 «Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы».

Согласно ГОСТ 34.602–2020 техническое задание должно включать следующие разделы:

- 1 Введение.
- 2 Основания для разработки.
- 3 Назначение приложения.
- 4 Требования к веб-приложению.
- 5 Требования к техническому обеспечению.
- 6 Требования к программному обеспечению.
- 7 Требования к тестированию.
- 8 Организационно – технические требования.

Техническое задание на разработку веб-приложения представлено в приложении А.

					ДП.09.02.7-1.26.221.03 ПЗ	Лист
						15
Изм.	Лист	№ докум.	Подпись	Дата		

3 Проектирование программного продукта

3.1 Архитектура программного продукта

Архитектурой веб-приложения является клиент-серверная.

Клиент-серверная архитектура – это модель организации вычислительных систем, в которой задачи распределены между клиентами и серверами. На рисунке 1 представлена схема клиент-серверной архитектуры.

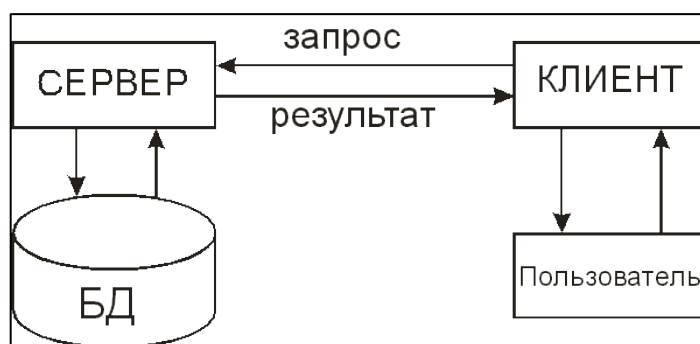


Рисунок 1 – Клиент-серверная архитектура

Основные компоненты клиент-серверной архитектуры:

- Клиенты – это устройства или приложения, которые запрашивают информацию или услуги у сервера. Клиенты могут быть как программными приложениями, так и аппаратными устройствами.

- Серверы – это компьютеры, которые предоставляют запрашиваемую информацию или услуги клиентам. Серверы могут выполнять различные функции, такие как хранение данных, обработка запросов, вычисления.

- Протоколы обмена данными – это правила или наборы инструкций, которые определяют, как клиенты и серверы обмениваются информацией.

Для серверной части веб-приложения будет использоваться python фреймворк Django. Данный фреймворк использует архитектурный шаблон MVT. На рисунке 2 представлен архитектурный шаблон MVT.

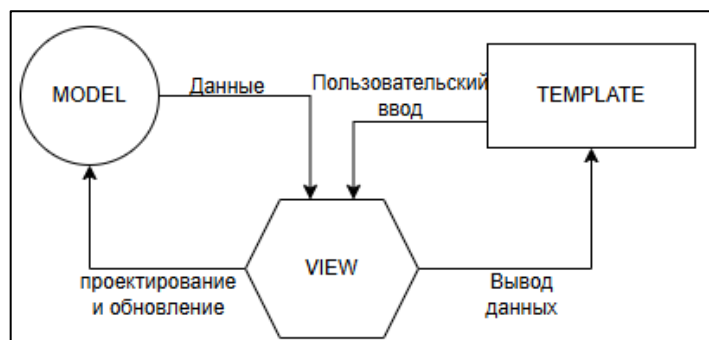


Рисунок 2 – Архитектурный шаблон MVC

MVT – это шаблон программирования, который позволяет разделить логику приложения на три части:

- Model (модель). Уровень данных, который определяет структуру базы данных и взаимодействует с ней (ORM).
- View (представление). Логический уровень. Получает HTTP-запросы, обрабатывает (используя данные из моделей) и возвращает HTTP-ответы (шаблоны).
- Template (шаблоны). Уровень представления (интерфейс). Это HTML-файлы, которые Django наполняет данными для отображения пользователю.

3.2 Функциональное и структурное проектирование

Диаграмма прецедентов в методологии Uses CASE – это инструмент для визуализации и описания функциональности программного продукта с точки зрения пользователей и внешних систем. Диаграмма позволяет определить, какие действия может выполнять веб-приложение и какие результаты может предоставить. Диаграмма прецедентов представлена на рисунке 3.

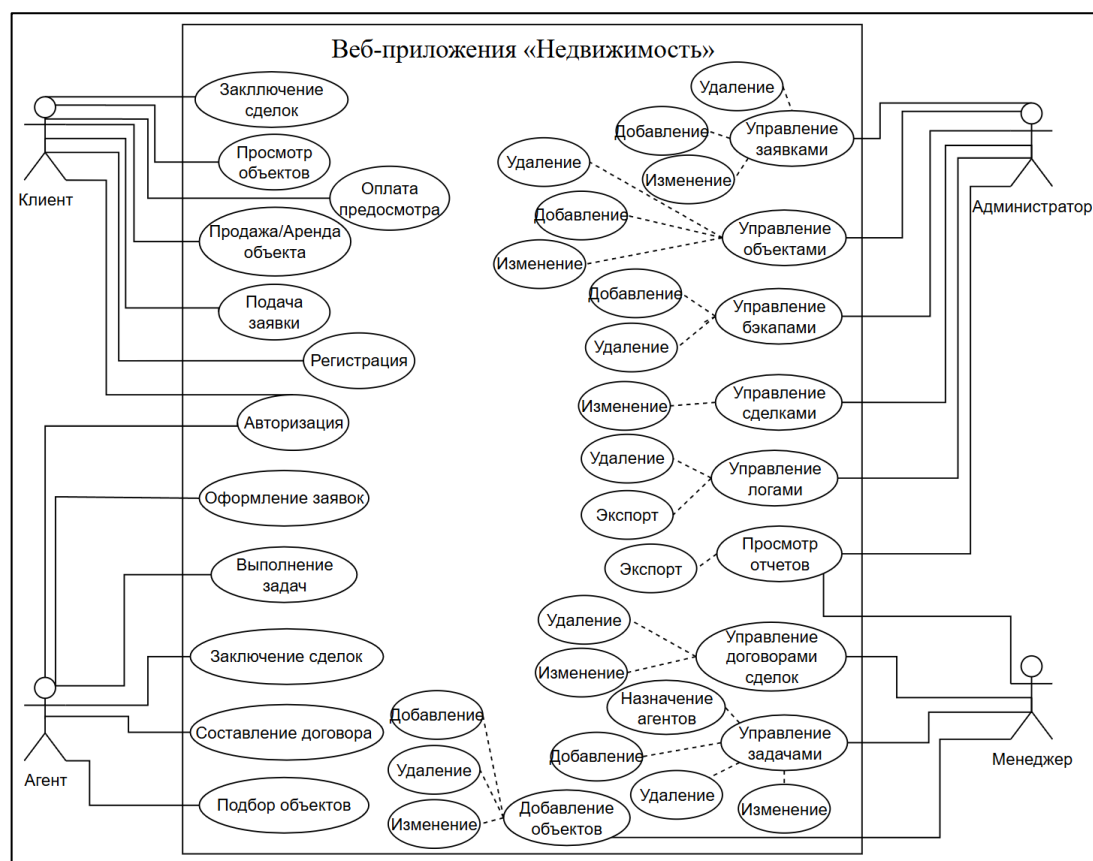


Рисунок 3 – Диаграмма прецедентов

На диаграмме представлены четыре основных прецедента:

- клиент: может зарегистрироваться или авторизоваться, просматривать каталог объектов недвижимости и выставлять свои объекты, подавать заявки на подбор/покупку, вносить оплату за предосмотр и участвовать в заключении сделок;
- агент: после авторизации выполняет ключевые рабочие процессы: оформляет заявки клиентов, работает с назначенными задачами, составляет договоры и проводит заключение сделок;
- менеджер: имеет доступ к просмотру аналитических отчётов, также добавляет, редактирует и изменяет объекты и также управляет задачи агентов;
- администратор: обладает расширенными правами администрирования системы, включая управление объектами недвижимости и заявками, контроль сделок и договоров, администрирование системных логов и управление резервными копиями (бекапами) данных, управляет кадровым составом.

Диаграмма показывает, что веб-приложение разделено на клиентскую часть и административную часть.

Диаграмма потоков данных показывает движение данных в веб-приложении откуда данные поступают, как обрабатываются, где хранятся и кому передаются. Диаграмма изображена на рисунке 4.

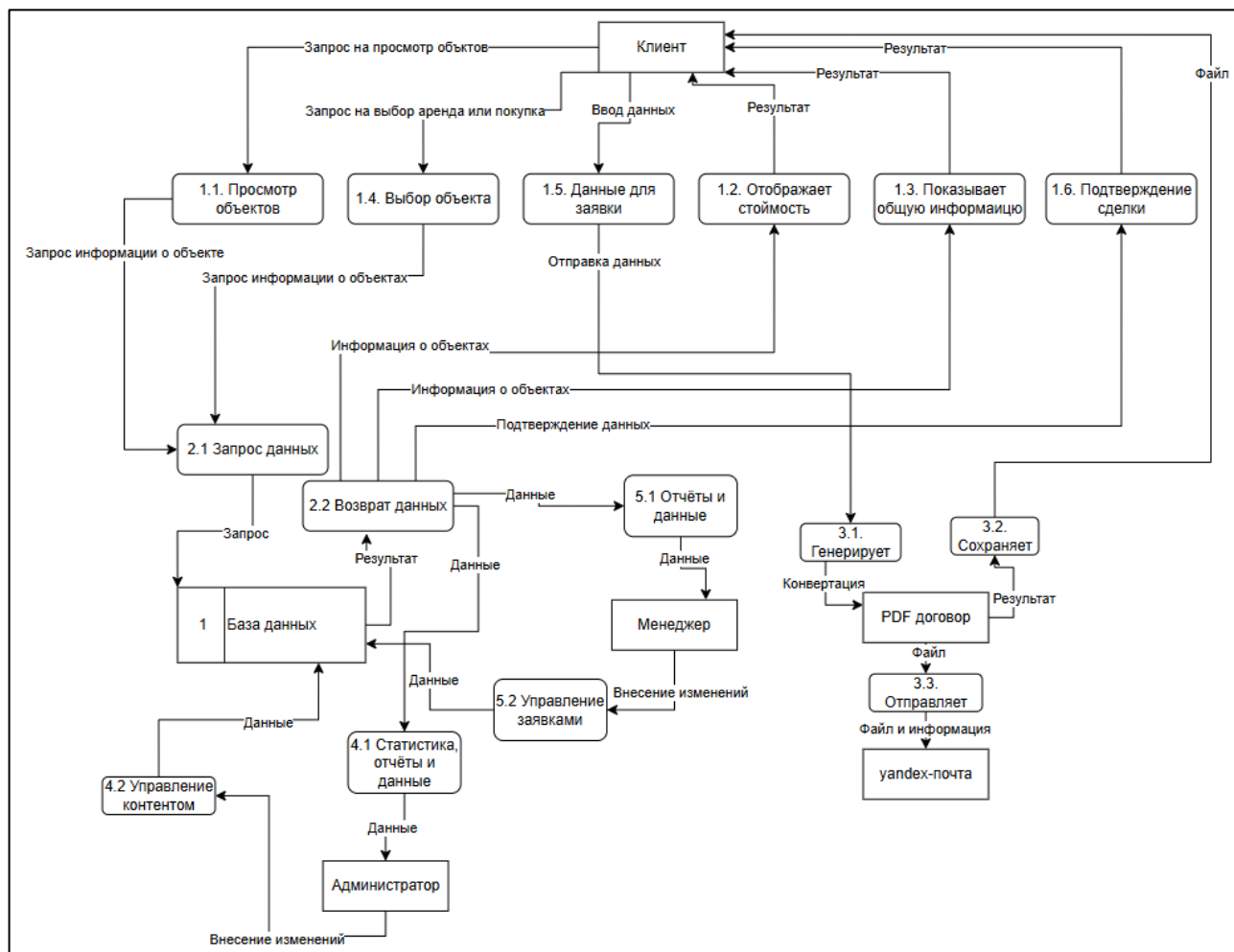


Рисунок 4 –Диаграмма потоков данных

Диаграмма иллюстрирует как информация обрабатывается и сохраняется в базе данных и возвращается в виде заявки и готового договора.

Созданные диаграммы для веб-приложения «Недвижимость» помогают визуализировать архитектуру и основные процессы. Диаграммы в совокупности обеспечивают полное представление о функционале, структуре и взаимодействиях

в веб-приложении, помогая разработчикам и заинтересованным сторонам эффективно управлять разработкой.

3.3 Проектирование базы данных

Перед началом разработки программного обеспечения важно спроектировать базу данных, определив, с какими данными будут работать пользователи веб-приложения и как эти данные взаимосвязаны. Инфологическая модель базы данных представлена в приложении Б.

Инфологическая модель представляет собой семантическое описание предметной области в терминах сущностей и отношений:

- RENOVATION_TYPE – сущность «Тип ремонта» с кодом и названием (без ремонта, косметический, евроремонт, дизайнерский).

- BATHROOM_TYPE – сущность «Тип санузла» с кодом и названием (совмещенный, отдельный).

- BUILDING_MATERIAL – сущность «Материал здания» с кодом и названием (кирпич, панель, монолит, блок, дерево).

- COMMERCIAL_PROPERTY_TYPE – сущность «Тип коммерческой недвижимости» с кодом и названием (офис, торговое помещение, склад, производственное помещение, свободное назначение, бизнес, коммерческий участок).

- AMENITY – сущность «Удобство» с кодом и названием (интернет, кондиционер, телевизор, холодильник, стиральная машина, посудомоечная машина, охрана, домофон, мебель, парковка).

- OPERATION_TYPE – сущность «Тип операции» с кодом и названием для классификации операций с недвижимостью (продажа, аренда).

- PROPERTY_STATUS – сущность «Статус объекта недвижимости» с кодом и названием состояния объекта (активно, продано, в архиве).

- REQUEST_STATUS – сущность «Статус заявки» с кодом и названием состояния заявки (новая, в работе, закрыта).
- DEAL_STATUS – сущность «Статус сделки» с кодом, названием (черновик, переговоры, в оформлении, завершена, отменена).
- TASK_STATUS – сущность «Статус задачи» с кодом, названием и порядком отображения (новая, в работе, выполнена, отменена).
- PROPERTY_TYPE – сущность «Тип недвижимости» с кодом и названием (квартира, дом/коттедж, коммерческая, земельный участок, гараж, комната).
- TASK_PRIORITY – сущность «Приоритет задачи» с кодом и названием (низкий, средний, высокий, срочный).
- TASK_TYPE – сущность «Тип задачи» с кодом и названием (позвонить клиенту, показать объект, подготовить документы, провести переговоры, подписать договор, другое).
- CLIENT_KIND – сущность «Вид клиента» с кодом и названием (физическое лицо, юридическое лицо).
- CONTRACT_STATUS – сущность «Статус договора» с кодом и названием (черновик, на согласовании, подписан, отклонен).
- USER_ROLE – сущность «Роль пользователя» с кодом, названием и описанием (клиент, администратор, агент, менеджер).
- AUDIT_ENTITY_TYPE – сущность «Тип сущности для аудита» с кодом и названием (объект недвижимости, заявка, задача, сделка, пользователь).
- AUDIT_ACTION – сущность «Действие для аудита» с кодом и названием (создан, обновлен, удален, статус изменен, назначен, завершен).
- REQUEST_MATCH_STATUS – сущность «Статус подборки объекта к заявке» с кодом и названием (предложен, просмотрен, одобрен клиентом, отклонен клиентом, снят с рассмотрения).
- DOCUMENT_TYPE – сущность «Тип документа» с кодом и названием (договор оказания услуг, договор купли-продажи, договор аренды, акт приема-

передачи, агентский договор, доверенность, расписка, дополнительное соглашение).

– DEAL_PARTICIPANT_ROLE – сущность «Роль участника сделки» с кодом и названием (покупатель, продавец, арендатор, арендодатель, поручитель, созаемщик, представитель, наследник, доверенное лицо, другое).

– VIEWING_STATUS – сущность «Статус просмотра» с кодом и названием (запланирован, подтвержден, идет просмотр, завершен, отменен, клиент не пришел).

– VIEWING_PAYMENT_STATUS – сущность «Статус оплаты просмотра» с кодом и названием (ожидает оплаты, оплачен, ошибка оплаты, возвращён).

– CITY – сущность «Город» с названием, регионом и внешним идентификатором адресного реестра.

– STREET – сущность «Улица» с названием, типом улицы, привязкой к городу и внешним идентификатором.

– HOUSE – сущность «Дом» с номером дома, широтой, долготой, почтовым индексом, привязкой к улице и внешним идентификатором (выступает в роли адресной привязки).

– USER – сущность «Пользователь» с данными для авторизации (email, хеш пароля), контактной информацией (телефон), ролью и флагами подтверждения (email, активность), датой последнего входа.

– EMPLOYEE_PROFILE – сущность «Профиль сотрудника» с персональными данными (имя, фамилия), внутренним телефоном и датой найма.

– CLIENT_PROFILE – сущность «Профиль клиента» с персональными данными (имя, фамилия, отчество) и видом клиента.

– CLIENT_INDIVIDUAL_DETAILS – сущность «Паспортные данные клиента-физлица» с серией, номером, наименованием выдавшего органа, датой выдачи и кодом подразделения.

– CLIENT_COMPANY_DETAILS – сущность «Реквизиты клиента-юрлица» с названием компании, ИНН, ОГРН, КПП и юридическим адресом.

					ДП.09.02.7-1.26.221.03 ПЗ	Лист
						22
Изм.	Лист	№ докум.	Подпись	Дата		

– PROPERTY – сущность «Объект недвижимости» с характеристиками (название, цена, общая площадь, количество комнат, этаж, координаты, кадастровый номер, описание), привязкой к дому, типу недвижимости, типу операции и статусу, флагами публикации и датами публикации/снятия с публикации.

– PROPERTY_PRICE_HISTORY – сущность «История изменения цены объекта» с привязкой к объекту, старой и новой ценой, инициатором изменения и датой.

– PROPERTY_OWNER – связующая сущность «Владелец объекта» для реализации связи между объектами недвижимости и клиентами-владельцами, с долей владения в процентах.

– BUILDING_DETAILS – сущность «Детали здания» с привязкой к дому, годом постройки, количеством этажей в доме, материалом здания и количеством лифтов.

– PROPERTY_DETAILS – сущность «Расширенные характеристики объекта» с привязкой к объекту, жилой площадью, площадью кухни, высотой потолков, количеством балконов, санузлов, спален, этажей в объекте, площадью участка, типом санузла и типом ремонта.

– COMMERCIAL_PROPERTY_DETAILS – сущность «Детали коммерческой недвижимости» с привязкой к объекту, типом коммерции, полезной площадью, высотой потолков, нагрузкой на пол, электрической мощностью, признаками отдельного входа, витринных окон, первой линии и количеством парковочных мест.

– PROPERTY_AMENITY – связующая сущность «Удобство объекта» для реализации связи между объектами недвижимости и удобствами.

– PROPERTY_PHOTO – сущность «Фотография объекта» с привязкой к объекту, URL-адресом изображения, подписью, признаком скрытия и порядком отображения.

					ДП.09.02.7-1.26.221.03 ПЗ	Лист
						23
Изм.	Лист	№ докум.	Подпись	Дата		

– PROPERTY_DOCUMENT – сущность «Документ объекта» с привязкой к объекту, названием документа, ссылкой на файл, информацией о верификации, пользователе, проверившем документ, и датой проверки.

– PROPERTY_VIEWING – сущность «Просмотр объекта» с привязкой к объекту, клиенту, сотруднику, запланированной датой просмотра, статусом и комментарием.

– VIEWING_PAYMENT – сущность «Оплата просмотра» с привязкой к просмотру, клиенту, объекту, сумме оплаты, статусом оплаты (ссылка на справочник статусов), идентификаторами заказа и транзакции Сбербанка, ссылкой на оплату и датой фактической оплаты.

– PAYMENT_HISTORY – сущность «История оплаты просмотра» с привязкой к платежу, предыдущим и новым статусом, комментарием, ответом от платёжной системы, инициатором изменения и датой создания.

– REQUEST – сущность «Заявка клиента» с критериями поиска (тип недвижимости, минимальная и максимальная цена, минимальная и максимальная площадь, количество комнат, предпочтительный город, район, пожелания по адресу, описание), привязкой к клиенту, сотруднику, объекту, типу операции и статусу.

– REQUEST_PROPERTY_MATCH – связующая сущность «Вариант объекта по заявке» для реализации связи между заявками и объектами, с привязкой к сотруднику, статусом подборки, заметкой агента и датой подтверждения.

– DEAL – сущность «Сделка» с номером, привязкой к объекту, сотруднику, типу операции, статусу сделки и заявке, финансовыми показателями (итоговая цена, процент комиссии), датой сделки, примечаниями и статусом договора.

– DEAL_PARTICIPANT – связующая сущность «Участник сделки» для реализации связи между сделками и клиентами с указанием роли участника (покупатель, продавец, арендатор, арендодатель).

– DEAL_DOCUMENT – сущность «Документ сделки» с привязкой к сделке, типом документа, ссылкой на файл, номером документа, путем к шаблону, датой генерации и пользователем, сгенерировавшим документ.

– TASK – сущность «Задача» с названием, описанием, приоритетом, типом, статусом, исполнителем, создателем, привязкой к клиенту/объекту/заявке/сделке, сроком выполнения, результатом и флагом автозакрытия.

– AUDIT_LOG – сущность «Запись журнала действий» с типом и ID сущности, кодом действия, сообщением, метаданными, инициатором и датой создания.

– DATABASE_BACKUP – сущность «Резервная копия БД» с именем файла, путем к файлу, размером, названием БД, СУБД, инструментом, создателем и датой создания.

- Связи между сущностями:
- Роль пользователя определяет права нескольких пользователей (1:M).
- Пользователь (сотрудник) имеет один профиль сотрудника (1:1).
- Пользователь (клиент) имеет один профиль клиента (1:1).
- Профиль клиента (физлицо) имеет одни паспортные данные (1:1).
- Профиль клиента (юрлицо) имеет одни реквизиты компании (1:1).
- Город содержит несколько улиц (1:M).
- Улица содержит несколько домов (1:M).
- Дом связан с несколькими объектами недвижимости (1:M).
- Дом имеет одни детальные характеристики здания (1:1).
- Тип операции классифицирует несколько объектов недвижимости (1:M).
- Статус объекта недвижимости определяет состояние нескольких объектов (1:M).
- Тип недвижимости классифицирует несколько объектов недвижимости (1:M).
- Объект недвижимости имеет одну запись расширенных характеристик (1:1).
- Объект недвижимости может иметь одну запись деталей коммерческой недвижимости (1:1).

- Объект недвижимости имеет несколько удобств через связующую сущность (1:M).
- Объект недвижимости имеет несколько фотографий (1:M).
- Объект недвижимости имеет несколько документов (1:M).
- Объект недвижимости имеет историю изменения цен (1:M).
- Объект недвижимости имеет несколько запланированных просмотров (1:M).
- Объект недвижимости имеет несколько владельцев через связующую сущность (1:M).
- Статус заявки определяет состояние нескольких заявок (1:M).
- Пользователь (клиент) может создавать несколько заявок (1:M).
- Пользователь (сотрудник) может обрабатывать несколько заявок (1:M).
- Объект недвижимости может быть напрямую связан с несколькими заявками (1:M).
- Тип операции используется в нескольких заявках (1:M).
- Город может быть предпочтительным для нескольких заявок (1:M).
- Заявка может иметь несколько предложенных вариантов объектов через связующую сущность (1:M).
- Объект недвижимости может быть предложен в нескольких вариантах по разным заявкам (1:M).
- Статус подборки определяет состояние нескольких вариантов объектов (1:M).
- Объект недвижимости участвует в нескольких сделках (1:M).
- Пользователь (сотрудник) сопровождает несколько сделок (1:M).
- Статус сделки определяет состояние нескольких сделок (1:M).
- Статус договора определяет состояние договоров в нескольких сделках (1:M).
- Заявка может быть связана с одной сделкой (1:1).
- Сделка имеет несколько участников через связующую сущность (1:M).

- Клиент может участвовать в нескольких сделках через связующую сущность (1:M).
- Клиент может быть связан с несколькими записями о владении объектами недвижимости (1:M).
- Запись о владении объектом недвижимости связывает клиента с объектом недвижимости (M:1).
- Объект недвижимости может иметь несколько записей о владении (1:M).
 - Роль участника сделки классифицирует несколько участников (1:M).
 - Сделка имеет несколько документов (1:M).
 - Тип документа классифицирует несколько документов сделки (1:M).
 - Приоритет задачи классифицирует несколько задач (1:M).
 - Тип задачи классифицирует несколько задач (1:M).
 - Статус задачи определяет состояние нескольких задач (1:M).
 - Пользователь может быть назначен исполнителем нескольких задач (1:M).
 - Пользователь может создавать несколько задач (1:M).
 - Объект недвижимости может быть связан с несколькими задачами (1:M).
 - Заявка может быть связана с несколькими задачами (1:M).
 - Сделка может быть связана с несколькими задачами (1:M).
 - Клиент может быть связан с несколькими задачами (1:M).
 - Пользователь может быть инициатором нескольких записей в журнале действий (1:M).
 - Тип сущности для аудита классифицирует несколько записей журнала (1:M).
 - Действие для аудита классифицирует несколько записей журнала (1:M).
 - Пользователь может создавать несколько резервных копий базы данных (1:M).
 - Статус просмотра определяет состояние нескольких просмотров (1:M).
 - Тип ремонта классифицирует характеристики нескольких объектов (1:M).
 - Тип санузла классифицирует характеристики нескольких объектов (1:M).
 - Материал здания классифицирует детали нескольких зданий (1:M).

- Тип коммерческой недвижимости классифицирует детали нескольких объектов (1:M).
- Удобство может быть привязано к нескольким объектам через связующую сущность (M:N).
- Статус оплаты просмотра определяет состояние нескольких оплат просмотров (1:M).
- Просмотр объекта может иметь не более одной записи оплаты (1:1).
- Оплата просмотра может иметь несколько записей в истории изменений своего статуса (1:M).

ER-модель базы данных представлена в приложении В.

Представленная ER-модель соответствует третьей нормальной форме:

1. Первая нормальная форма: все атрибуты атомарны.
2. Вторая нормальная форма: нет частичных зависимостей от составного ключа.
3. Третья нормальная форма: нет транзитивных зависимостей – все неключевые атрибуты зависят только от первичного ключа.

Обоснование выбора 3NF:

- Устраняет избыточность данных.
- Обеспечивает целостность данных.
- Упрощает поддержку и модификацию БД.
- Оптимальна для OLTP-систем.

База данных веб-приложения состоит из 51 таблиц и описана в таблицах с 5 до 55.

Таблица 5 – Таблица «renovation_types»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(25)	Код типа (none, cosmetic, euro, designer)
name	VARCHAR(100)	Название на русском

Таблица 6 – Таблица «bathroom_types»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(25)	Код типа (combined, separate)
name	VARCHAR(50)	Название на русском

Таблица 7 – Таблица «building_materials»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(25)	Код материала (brick, panel, monolith, block, wood)
name	VARCHAR(100)	Название на русском

Таблица 8 – Таблица «commercial_property_types»

Атрибут	Тип данных	Описание
id	INT (PK, AI)	Уникальный идентификатор
code	VARCHAR(25)	Код типа (office, retail, warehouse)
name	VARCHAR(50)	Название на русском

Таблица 9 – Таблица «amenities»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(25)	Код удобства (internet, parking)
name	VARCHAR(50)	Название на русском

Таблица 10 – Таблица «operation_types»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(25)	Код операции (sale, rent)
name	VARCHAR(50)	Название на русском

Таблица 11 – Таблица «property_statuses»

Атрибут	Тип данных	Описание
id	INT (PK, AI)	Уникальный идентификатор
code	VARCHAR(25)	Код статуса (active, sold, archived)
name	VARCHAR(50)	Название на русском

Таблица 12 – Таблица «request_statuses»

Атрибут	Тип данных	Описание
id	INT (PK, AI)	Уникальный идентификатор
code	VARCHAR(25)	Код статуса (new, in_progress, closed)
name	VARCHAR(50)	Название на русском

Таблица 13 – Таблица «deal_statuses»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(25)	Код статуса
name	VARCHAR(50)	Название на русском
order	SMALLINT	Порядок статуса в цепочке

Таблица 14 – Таблица «task_statuses»

Атрибут	Тип данных	Описание
id	INT (PK, AI)	Уникальный идентификатор
code	VARCHAR(20)	Код статуса
name	VARCHAR(50)	Название на русском
order	SMALLINT	Порядок статуса в цепочке

Таблица 15 – Таблица «property_types»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(25)	Код типа (apartment, house, commercial, land, garage, room)
name	VARCHAR(50)	Название на русском

Таблица 16 – Таблица «task_priorities»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(25)	Код приоритета (low, medium, high, urgent)
name	VARCHAR(50)	Название на русском

Таблица 17 – Таблица «task_types»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(25)	Код типа задачи
name	VARCHAR(50)	Название на русском

Таблица 18 – Таблица «client_kinds»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(25)	Код вида (individual, company)
name	VARCHAR(50)	Название на русском

Таблица 19 – Таблица «contract_statuses»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(25)	Код типа задачи

Продолжение таблицы 19

Атрибут	Тип данных	Описание
name	VARCHAR(50)	Название на русском

Таблица 20 – Таблица «user_roles»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(25)	Код роли (client, admin, agent, manager)
name	VARCHAR(50)	Название на русском
description	TEXT	Описание роли

Таблица 21 – Таблица «audit_entity_types»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(25)	Код сущности (property, request, task, deal, user)
name	VARCHAR(50)	Название на русском

Таблица 22 – Таблица «audit_actions»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(25)	Код действия (created, updated, deleted)
name	VARCHAR(50)	Название на русском

Таблица 23 – Таблица «request_match_statuses»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(25)	Код статуса (proposed, viewed, approved, rejected, withdrawn)
name	VARCHAR(50)	Название на русском

Таблица 24 – Таблица «document_types»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(25)	Код типа (service_contract, sale_contract)
name	VARCHAR(50)	Название на русском

Таблица 25 – Таблица «deal_participant_roles»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(25)	Код роли (buyer, seller, tenant, landlord, guarantor)

Продолжение таблицы 25

Атрибут	Тип данных	Описание
name	VARCHAR(50)	Название на русском

Таблица 26 – Таблица «viewing_statuses»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(20)	Код статуса (scheduled, confirmed, in_progress, completed, cancelled, no_show)
name	VARCHAR(100)	Название на русском

Таблица 27 – Таблица «cities»

Поле	Тип данных	Описание
id	INT	Уникальный идентификатор города
name	VARCHAR(100)	Название города
region	VARCHAR(100)	Регион или область
external_id	CHAR(36)	Внешний идентификатор

Таблица 28 – Таблица «streets»

Поле	Тип данных	Описание
id	INT	Уникальный идентификатор улицы
cities_id	INT	Ссылка на город
name	VARCHAR(150)	Название улицы
street_type	VARCHAR(20)	Тип улицы
external_id	CHAR(36)	Внешний идентификатор

Таблица 29 – Таблица «houses»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
street_id	INT	Ссылка на улицу
house_number	VARCHAR(20)	Номер дома
postal_code	VARCHAR(10)	Почтовый индекс
coordinates_lat	DECIMAL(10,8)	Широта
coordinates_lon	DECIMAL(11,8)	Долгота
cadastral_number	VARCHAR(50)	Кадастровый номер
external_id	CHAR(36)	Внешний идентификатор

Таблица 90 – Таблица «users»

Атрибут	Тип данных	Описание
id	INT (PK, AI)	Уникальный идентификатор
email	VARCHAR(100)	Email (для входа)
password	VARCHAR(128)	Хэш пароля
phone	VARCHAR(20)	Телефон

Продолжение таблицы 30

Атрибут	Тип данных	Описание
role_id	INT	Роль пользователя
is_active	TINYINT(1)	Флаг активности
is_email_verified	TINYINT(1)	Email подтвержден
last_login	DATETIME	Дата последнего входа
created_at	DATETIME	Дата создания

Таблица 31 – Таблица «employee_profiles»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
user_id	INT	Ссылка на пользователя
first_name	VARCHAR(50)	Имя
last_name	VARCHAR(50)	Фамилия
hire_date	DATE	Дата приема на работу
internal_phone	VARCHAR(20)	Внутренний телефон
created_at	DATETIME	Дата создания
updated_at	DATETIME	Дата обновления

Таблица 32 – Таблица «client_profiles»

Атрибут	Тип данных	Описание
id	INT (PK, AI)	Уникальный идентификатор
user_id	INT	Ссылка на пользователя
first_name	VARCHAR(50)	Имя
last_name	VARCHAR(50)	Фамилия
middle_name	VARCHAR(50)	Отчество
client_kind_ref_id	INT	Вид клиента
created_at	DATETIME	Дата создания
updated_at	DATETIME	Дата обновления

Таблица 33 – Таблица «client_individual_details»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
profile_id	INT	Ссылка на профиль
passport_series	VARCHAR(4)	Серия паспорта
passport_number	VARCHAR(6)	Номер паспорта
passport_issued_by	VARCHAR(255)	Кем выдан
passport_issued_date	DATE	Дата выдачи
passport_code	VARCHAR(7)	Код подразделения
created_at	DATETIME	Дата создания
updated_at	DATETIME	Дата обновления

Таблица 34 – Таблица «client_company_details»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор

Продолжение таблицы 34

Атрибут	Тип данных	Описание
profile_id	INT	Ссылка на профиль
company_name	VARCHAR(255)	Название компании
company_inn	VARCHAR(10)	ИНН
company_ogrn	VARCHAR(13)	ОГРН
company_kpp	VARCHAR(9)	КПП
legal_address	TEXT	Юридический адрес
created_at	DATETIME	Дата создания
updated_at	DATETIME	Дата обновления

Таблица 35 – Таблица «properties»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
title	VARCHAR(255)	Заголовок объявления
operation_type_id	INT	Тип операции
status_id	INT	Статус объекта
house_id	INT	Ссылка на дом
property_type_ref_id	INT	Тип недвижимости
price	DECIMAL(15,2)	Цена объекта
area_total	DECIMAL(8,2)	Общая площадь
rooms_count	INT	Количество комнат
description	TEXT	Описание объекта
floor_number	INT	Номер этажа
is_published	TINYINT(1)	Флаг публикации
published_at	DATETIME	Дата публикации
unpublished_at	DATETIME	Дата снятия с публикации
created_at	DATETIME	Дата создания
updated_at	DATETIME	Дата обновления

Таблица 36 – Таблица «property_price_history»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
property_id	INT	Ссылка на объект
old_price	DECIMAL(15,2)	Старая цена
new_price	DECIMAL(15,2)	Новая цена
changed_by_id	INT	Кто изменил
changed_at	DATETIME	Дата изменения

Таблица 37 – Таблица «property_owners»

Атрибут	Тип данных	Описание
property_id	INT	Ссылка на объект
client_profile_id	INT	Ссылка на владельца
ownership_share	DECIMAL(5,2)	Доля владения (%)
created_at	DATETIME	Дата добавления

Таблица 38 – Таблица «building_details»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
house_id	INT	Ссылка на дом
year_built	SMALLINT	Год постройки
total_floors	SMALLINT	Количество этажей в доме
building_material_id	INT NULL	Материал здания
elevators_count	SMALLINT	Количество лифтов

Таблица 39 – Таблица «property_details»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
property_id	INT	Ссылка на объект
living_area	DECIMAL(8,2)	Жилая площадь
kitchen_area	DECIMAL(8,2)	Площадь кухни
ceiling_height	DECIMAL(4,2)	Высота потолков
balcony_count	TINYINT	Количество балконов
bathroom_count	TINYINT	Количество санузлов
bathroom_type_id	INT	Тип санузла
renovation_type_id	INT	Тип ремонта
bedrooms_count	SMALLINT	Количество спален
floors_count	SMALLINT	Количество этажей в объекте
land_area	DECIMAL(10,2)	Площадь участка (м²)

Таблица 40 – Таблица «commercial_property_details»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
property_id	INT	Ссылка на объект
commercial_type_id	INT	Тип коммерции
usable_area	DECIMAL(10,2)	Полезная площадь
ceiling_height	DECIMAL(4,2)	Высота потолков
floor_load	DECIMAL(10,2)	Нагрузка на пол
electric_power_kw	DECIMAL(10,2)	Электрическая мощность (кВт)
has_separate_entrance	TINYINT(1)	Отдельный вход
has_display_windows	TINYINT(1)	Витринные окна
is_first_line	TINYINT(1)	Первая линия
parking_spaces	SMALLINT	Парковочные места

Таблица 41 – Таблица «property_amenities»

Атрибут	Тип данных	Описание
property_id	INT	Ссылка на объект
amenity_id	INT	Ссылка на удобство

Таблица 42 – Таблица «property_photos»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
property_id	INT	Ссылка на объект
url	TEXT	URL изображения
caption	VARCHAR(255)	Подпись к фото
is_hidden	TINYINT(1)	Флаг скрытия
order	INT	Порядок сортировки
uploaded_at	DATETIME	Дата загрузки

Таблица 43 – Таблица «property_documents»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
property_id	INT	Ссылка на объект
document_name	VARCHAR(100)	Название документа
file_url	TEXT	URL файла
is_verified	TINYINT(1)	Флаг проверки
verified_by_id	INT	Кто проверил
verified_at	DATETIME	Дата проверки
uploaded_at	DATETIME	Дата загрузки

Таблица 44 – Таблица «property_viewings»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
property_id	INT	Ссылка на объект
client_profile_id	INT	Ссылка на клиента
employee_profile_id	INT	Ссылка на сотрудника
viewing_date	DATETIME	Дата и время просмотра
status_id	INT	Статус просмотра
comment	TEXT	Комментарий
created_at	DATETIME	Дата создания

Таблица 45 – Таблица «requests»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
client_profile_id	INT	Ссылка на клиента
employee_profile_id	INT	Ссылка на агента
property_id	INT	Ссылка на объект
operation_type_id	INT	Тип операции
status_id	INT	Статус заявки
property_type_id	INT	Желаемый тип недвижимости
preferred_city_id	INT	Предпочтительный город
preferred_district	VARCHAR(100)	Предпочтительный район
max_area	DECIMAL(8,2)	Максимальная площадь
rooms_count	INT	Желаемое количество комнат

Продолжение таблицы 45

Атрибут	Тип данных	Описание
address_preferences	TEXT	Текстовые предпочтения по адресу
description	TEXT	Описание требований
created_at	DATETIME	Дата создания
updated_at	DATETIME	Дата обновления
closed_at	DATETIME	Дата закрытия

Таблица 46 – Таблица «request_property_matches»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
request_id	INT	Ссылка на заявку
property_id	INT	Ссылка на объект
employee_profile_id	INT	Ссылка на сотрудника
status_id	INT	Статус подборки
agent_note	TEXT	Заметка агента
confirmed_at	DATETIME	Дата подтверждения
confirmed_by_id	INT	Кто подтвердил
created_at	DATETIME	Дата создания

Таблица 47 – Таблица «deals»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
deal_number	VARCHAR(50)	Номер сделки
property_id	INT	Ссылка на объект
employee_profile_id	INT	Ссылка на сотрудника
operation_type_id	INT	Тип операции
status_id	INT	Статус сделки
request_id	INT	Ссылка на заявку
price_final	DECIMAL(15,2)	Итоговая цена
commission_percent	DECIMAL(10,2)	Процент комиссии
deal_date	DATE	Дата сделки
notes	TEXT	Примечания
contract_status_ref_id	INT	Статус договора
created_at	DATETIME	Дата создания
updated_at	DATETIME	Дата обновления

Таблица 48 – Таблица «deal_participants»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
deal_id	INT	Ссылка на сделку
client_profile_id	INT	Ссылка на клиента
role_id	INT	Роль участника
created_at	DATETIME	Дата добавления

Таблица 49 – Таблица «deal_documents»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
deal_id	INT	Ссылка на сделку
document_type_id	INT	Тип документа
file_url	TEXT	URL файла
document_number	VARCHAR(50)	Номер документа
template_path	VARCHAR(255)	Путь к шаблону
generated_at	DATETIME	Дата генерации
generated_by_id	INT	Кто сгенерировал
created_at	DATETIME	Дата создания

Таблица 50 – Таблица «tasks»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
title	VARCHAR(255)	Заголовок задачи
description	TEXT	Описание
priority_ref_id	INT	Приоритет
task_type_ref_id	INT	Тип задачи
status_id	INT	Статус
assignee_id	INT	Исполнитель
created_by_id	INT	Создатель
client_profile_id	INT	Связанный клиент
property_id	INT	Связанный объект
request_id	INT	Связанная заявка
deal_id	INT	Связанная сделка
due_date	DATETIME	Срок выполнения
completed_at	DATETIME	Дата завершения
result	TEXT	Результат
is_auto_closed	TINYINT(1)	Флаг автозакрытия
created_at	DATETIME	Дата создания
updated_at	DATETIME	Дата обновления

Таблица 51 – Таблица «viewing_payments»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
viewing_id	INT	Ссылка на просмотр (уникальный)
client_id	INT	Ссылка на клиента-плательщика
property_id	INT	Ссылка на объект
amount	DECIMAL(10, 2)	Сумма оплаты
status_id	INT	Ссылка на справочник
sber_order_id	VARCHAR(128)	ID заказа Сбербанка
paid_at	DATETIME	Дата фактической оплаты

Таблица 52 – Таблица «viewing_payment_statuses»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
code	VARCHAR(25)	Статус оплаты (pending, paid, failed, refunded)
name	VARCHAR(50)	Название на русском

Таблица 53 – Таблица «payment_history»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
payment_id	INT	Ссылка на платёж
old_status	VARCHAR(16)	Предыдущий статус
new_status	VARCHAR(16)	Новый статус
sber_response	JSON	Ответ от платёжной системы
changed_by_id	INT	Кто изменил статус

Таблица 54 – Таблица «audit_logs»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
entity_type_id	INT	Тип сущности
entity_id	INT	ID сущности
action_id	INT	Код действия
message	TEXT	Сообщение
metadata	JSON	Метаданные
actor_id	INT	Кто выполнил действие
created_at	DATETIME	Дата создания

Таблица 55 – Таблица «database_backups»

Атрибут	Тип данных	Описание
id	INT	Уникальный идентификатор
filename	VARCHAR(255)	Имя файла
file	VARCHAR(100)	Путь к файлу
size_bytes	BIGINT	Размер в байтах
database_name	VARCHAR(255)	Имя базы данных
engine_label	VARCHAR(120)	Название движка
tool_label	VARCHAR(120)	Название инструмента
created_by_id	INT	Кто создал бэкап
created_at	DATETIME	Дата создания

Спроектированная база данных полностью соответствует требованиям предметной области для веб-приложения «Недвижимость». Для обеспечения целостности данных, автоматизации рутинных бизнес-процессов и оптимизации формирования аналитической отчетности на уровне СУБД PostgreSQL были

спроектированы и реализованы следующие программные объекты: триггеры, хранимые процедуры и представления (VIEW). Использование серверной логики позволяет снизить нагрузку на прикладной уровень (Django ORM), гарантировать выполнение критических правил независимо от источника изменения данных и ускорить формирование сводных показателей. Триггер автоматического закрытия задач при завершении сделки

В веб-приложении существует функциональная зависимость между сущностями «Сделка» (deals) и «Задача» (tasks). При переходе сделки в финальный статус «Завершена» все активные задачи, связанные с данной сделкой, должны быть автоматически переведены в статус «Выполнена», чтобы исключить накопление неактуальных поручений у сотрудников. Для реализации этой логики был создан триггер, который срабатывает после обновления статуса сделки. Триггерная функция проверяет изменение статуса и при необходимости обновляет записи в таблице задач, фиксируя дату завершения и устанавливая флаг автоматического закрытия. Код триггера представлен на рисунке 5.

```
CREATE OR REPLACE FUNCTION fn_auto_complete_tasks_on_deal_finish()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status_id = (SELECT id FROM deal_statuses WHERE code = 'completed')
    AND OLD.status_id != NEW.status_id THEN

        UPDATE tasks
        SET status_id = (SELECT id FROM task_statuses WHERE code = 'done'),
            completed_at = NOW(),
            is_auto_closed = TRUE
        WHERE deal_id = NEW.id
        AND status_id IN (SELECT id FROM task_statuses WHERE code IN ('new', 'in_progress'));

        INSERT INTO audit_logs (entity_type_id, entity_id, action_id, message, actor_id, created_at)
        VALUES (
            (SELECT id FROM audit_entity_types WHERE code = 'deal'),
            NEW.id,
            (SELECT id FROM audit_actions WHERE code = 'status_changed'),
            'Автоматическое закрытие связанных задач при завершении сделки',
            NULL,
            NOW()
        );
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_deal_status_change
AFTER UPDATE OF status_id ON deals
FOR EACH ROW EXECUTE FUNCTION fn_auto_complete_tasks_on_deal_finish();
```

Рисунок 5 – Триггер «Автоматического закрытия задачи»

Хранимая процедура генерации отчета по эффективности агентов. Для раздела «Отчетность» была разработана хранимая процедура. Данная процедура предназначена для агрегации данных о деятельности сотрудников агентства недвижимости за заданный временной период. Она вычисляет ключевые показатели эффективности: количество успешно заключенных сделок и общую сумму комиссионного вознаграждения каждого агента. Использование хранимой процедуры позволяет выполнять сложные запросы с объединением таблиц непосредственно на стороне сервера баз данных, что значительно сокращает время формирования отчета и объем передаваемых данных по сравнению с обработкой на уровне приложения. Код процедуры представлен на рисунке 6.

```
CREATE OR REPLACE PROCEDURE sp_generate_agent_performance_report(
    p_start_date DATE,
    p_end_date DATE,
    OUT r_username VARCHAR,
    OUT r_deals_count INT,
    OUT r_total_commission DECIMAL(15,2)
)
LANGUAGE plpgsql
AS $$
BEGIN
    FOR r_username, r_deals_count, r_total_commission IN
        SELECT
            u.username,
            COUNT(d.id) as deals_cnt,
            COALESCE(SUM(d.commission_amount), 0) as total_comm
        FROM deals d
        JOIN users u ON d.employee_profile_id = (SELECT id FROM employee_profiles WHERE user_id = u.id LIMIT 1)
        JOIN deal_statuses ds ON d.status_id = ds.id
        WHERE d.deal_date BETWEEN p_start_date AND p_end_date
            AND ds.code = 'completed'
        GROUP BY u.username
    LOOP
        RETURN NEXT;
    END LOOP;
END;
$$;
```

Рисунок 6 – Хранимая процедура «Генерации отчета по эффективности агентов»

Для формирования оперативной финансовой отчетности и отображения сводной статистики на панели администратора создано представление. Оно обеспечивает автоматическую агрегацию доходов по месяцам исключительно по успешно завершенным сделкам, что гарантирует достоверность финансовых показателей в реальном времени без дополнительной нагрузки на прикладной уровень. Это существенно повышает производительность интерфейса и снижает

нагрузку на СУБД при частых обновлениях главной страницы. Код представления представлен на рисунке 7.

```
CREATE OR REPLACE VIEW vw_deal_revenue_stats AS
SELECT
    DATE_TRUNC('month', d.deal_date) AS report_month,
    COUNT(d.id) AS total_deals_count,
    COALESCE(SUM(d.price_final), 0) AS total_revenue,
    COALESCE(AVG(d.commission_percent), 0) AS avg_commission_percent,
    COALESCE(SUM(d.commission_amount), 0) AS total_commission_income
FROM deals d
JOIN deal_statuses ds ON d.status_id = ds.id
WHERE ds.code = 'completed'
GROUP BY DATE_TRUNC('month', d.deal_date)
ORDER BY report_month DESC;
```

Рисунок 7 – Представление «финансовой отчётности»

ER-модель в третьей нормальной форме обеспечивает оптимальную структуру для хранения данных, минимизируя избыточность и обеспечивая целостность. Четко определенные связи между сущностями позволяют эффективно реализовать все бизнес-процессы веб-приложения, от просмотра объектов до совершения сделок. Выбранные типы данных и ограничения гарантируют корректное хранение информации и предотвращают возникновение противоречивых состояний в базе данных.

3.4 Проектирование пользовательского интерфейса

Для проектирования пользовательского интерфейса веб-приложения «Недвижимость» использовался онлайн-инструмент Draw.io. Данный инструмент был выбран за кроссплатформенность, простоту использования, широкий набор элементов для построения схем и удобство совместной работы, что позволило быстро и наглядно создать прототипы всех основных экранов.

Были разработаны прототипы всех ключевых окон веб-приложения, соответствующих функциональным требованиям, предъявляемым к клиентской и

административной частям приложения. Страница Авторизация изображена на рисунке 8.

Рисунок 8 – Страница «Авторизации»

Страница авторизации содержит стандартную форму для ввода учетных данных, кнопку для входа. Дизайн выполнен в минималистичном стиле, фокусирующем внимание на процессе аутентификации. Страница Регистрации изображена на рисунке 9.

Рисунок 9 – Страница «Регистрации»

Изм.	Лист	№ докум.	Подпись	Дата

Страница регистрации является входным узлом для новых пользователей. На странице регистрации представлена форма для заполнения учётных данных – имя, фамилия, адрес электронной почты, номер телефона и пароль. Страница обеспечивает создание новой учётной записи с автоматическим присвоением роли «клиент» и предоставляет ссылку для перехода к авторизации уже зарегистрированным пользователям. Страница «Главная страница» изображена на рисунке 10.



Рисунок 10 – Страница «Главная страница»

Главная страница является центральным узлом системы. На странице представлена сводная информация. Страница обеспечивает навигацию по основным разделам: объекты, заявки, сделки, задачи и отчёты – в зависимости от роли пользователя (клиент, агент, менеджер, администратор). Страница «Объекты недвижимости» изображена на рисунке 11.

Сводка
Объекты
Модерация
Заявки
Задачи
Сделки
Админ-панель
Аккаунт

Пользователь
Выход

ОБЪЕКТЫ
Импорт
Экспорт
Добавить объект

Каталог недвижимости

Фильтр объектов

Тип операции

Статус

Тип помещения

Комнат

Цена от

Цена до

Поиск

Дата отчёта от

Дата отчёта до

Сбросить

Применить

Сводка

КОЛИЧЕСТВО ОБЪЕКТОВ

Карточка объекта

Карточка объекта

Карточка объекта

Карточка объекта

Карточка объекта

Карточка объекта

Рисунок 11 – Страница «Объекты недвижимости»

Данная страница представляет собой основной рабочий интерфейс для управления каталогом недвижимости. В левой части расположен блок фильтрации и поиска, позволяющий отбирать объекты по типу операции (продажа/аренда), статусу объекта (активен, в резерве, продан, сдан), типу помещения (квартира, офис, склад), количеству комнат, диапазону цен, а также по дате создания или обновления. Также предусмотрено поле для быстрого текстового поиска по адресу или названию объекта. Внизу блока фильтрации расположены кнопки «Сбросить» и «Применить» для управления параметрами отбора. В верхней части страницы отображается сводная информация – количество объектов, соответствующих текущим критериям фильтрации. Основная часть экрана занята плиточным представлением карточек объектов, где для каждого объекта отображаются его

фотография, адрес, стоимость, количество комнат и статус. При нажатии на карточку открывается детальная информация об объекте. Интерфейс оптимизирован для быстрой навигации. Страница «Заявки клиентов» изображена на рисунке 12.

СводкаОбъектыМодерацияЗаявкиЗадачиСделкиАдмин-панельАккаунт

ПользовательВыход

Заявки

Новая заявка

Область просмотра

ВсеНеразобранныеМои

Фильтры

ОперацииСтатусСоздано отСоздано доСбросить

Список заявок

Экспорт

Информация о заявке	Редактировать	Удалить	Закрыть
Информация о заявке	Редактировать	Удалить	Закрыть
Информация о заявке	Редактировать	Удалить	Закрыть

Рисунок 12 – Страница «Заявки клиентов»

Данная страница представляет собой интерфейс для управления заявками клиентов на подбор, покупку или аренду недвижимости. В верхней части расположен блок фильтрации, позволяющий отбирать заявки по статусу. Кнопка «Новая заявка» обеспечивает переход к форме создания заявки. Основная часть экрана занята табличным представлением, где отображаются ключевые данные: информация о клиенте, желаемый объект недвижимости, текущий статус обработки, закреплённый агент-исполнитель и дата создания заявки. Колонка «Действия» предоставляет инструменты для перехода к детальному просмотру заявки, редактирования параметров, смены статуса или назначения ответственного

сотрудника. Интерфейс оптимизирован для эффективной работы менеджеров и агентов с портфелем клиентских заявок.

Интерфейс спроектирован интуитивно понятным и удобным для конечного пользователя, с четкой последовательностью.

Использование прототипа на раннем этапе позволило утвердить логику взаимодействия, визуальную структуру и навигацию до начала этапа разработки клиентской части, что способствовало снижению количества доработок на поздних стадиях проекта.

					ДП.09.02.7-1.26.221.03 ПЗ	Лист
						47
Изм.	Лист	№ докум.	Подпись	Дата		

4 Реализация программного продукта

4.1 Реализация базы данных

Разработка базы данных веб-приложения «Недвижимость» выполнялась с использованием встроенного ORM фреймворка Django. Такой подход позволил описывать структуру данных на языке Python с последующей генерацией SQL-кода и миграций, что обеспечивает независимость от конкретной СУБД и упрощает сопровождение проекта.

Все модели данных были реализованы в файле `key/models.py`. Общее количество моделей составило 51, что полностью покрывает все сущности предметной области, выявленные на этапе проектирования веб-приложения «Недвижимость». На рисунке 13 изображён фрагмент модели `User`.

```
class User(AbstractBaseUser, PermissionsMixin):
    USER_TYPE_CHOICES = [
        ('employee', 'Сотрудник'),
        ('client', 'Клиент'),
    ]
    username = models.CharField(max_length=50, unique=True)
    email = models.EmailField(max_length=255, unique=True)
    phone = models.CharField(max_length=20, unique=True, blank=True, null=True)

    user_type = models.CharField(max_length=20, choices=USER_TYPE_CHOICES,
                                default='client')
    role = models.ForeignKey(UserRole, on_delete=models.SET_NULL,
                            blank=True, null=True, related_name='users')
    is_active = models.BooleanField(default=True)
    is_staff = models.BooleanField(default=False)
    is_email_verified = models.BooleanField(default=False)
    is_phone_verified = models.BooleanField(default=False)

    last_login = models.DateTimeField(blank=True, null=True)
    last_ip = models.GenericIPAddressField(blank=True, null=True)

    created_at = models.DateTimeField(default=timezone.now)
    updated_at = models.DateTimeField(auto_now=True)
```

Рисунок 13 – Модели «User»

Для управления изменениями схемы базы данных использовался механизм миграций Django. Каждое изменение моделей сопровождалось созданием и применением миграций с помощью команд `python manage.py makemigrations` и `python manage.py migrate`.

База данных веб-приложения полностью реализована на уровне ORM Django. Создано 51 моделей, соответствующих инфологической и ER-моделям, разработанным на этапе проектирования веб-приложения управления недвижимостью. Все модели находятся в третьей нормальной форме, что позволяет исключить избыточность данных и обеспечить целостность информации.

В моделях реализованы связи между сущностями с использованием `ForeignKey`, автоматическая установка даты создания и обновления записей, а также валидация данных на уровне моделей.

4.2 Реализация серверной части

Серверная часть веб-приложения реализована на фреймворке Django с использованием архитектурного шаблона MVT (Model-View-Template). Разработка велась в интегрированной среде PyCharm Professional с использованием системы контроля версий Git.

Файл `views.py` содержит все контроллеры приложения. Общий объём файла составляет более 1000 строк кода, реализующих следующие группы функций:

- Аутентификация и управление пользователями – `RegisterView` (регистрация клиента с верификацией email), `VerifyEmailView` (подтверждение кода из письма), `LoginView` (JWT-токены), `MeView` (текущий пользователь), `UserViewSet` (список пользователей, экспорт, назначение ролей через `assign_role`).

- Работа с объектами недвижимости – `PropertyViewSet` (CRUD, смена статуса с проверкой правил перехода, массовое архивирование, история статусов, модерация объектов клиентов через `approve/reject`, импорт из CSV/XLSX, загрузка фото, автоматическое обогащение через «ДваГис»).

– Заявки клиентов и подборка объектов – RequestViewSet (создание, закрытие с выбором исхода, взятие в работу с проверкой лимитов, привязка и удаление вариантов, подтверждение подобранного объекта, массовое закрытие), RequestPropertyMatchViewSet (только чтение подборки).

– Сделки и договоры – DealViewSet (CRUD, генерация PDF-договора через фоновую очередь, повторная генерация, скачивание готового договора, экспорт сделок).

– Задачи сотрудников – TaskViewSet (CRUD, смена статуса, старт/пауза/завершение, запись шагов workflow, массовые операции: пауза, завершение, удаление, получение текущей задачи в работе, экспорт задач).

– Справочники и настройки – OperationTypeViewSet, PropertyStatusViewSet, RequestStatusViewSet, DealStatusViewSet, TaskStatusViewSet, UserRoleViewSet, CityViewSet, StreetViewSet, HouseViewSet, AddressViewSet.

– Фото, документы, история – PropertyPhotoViewSet (управление обложкой, порядок фото, скрывание/показ), PropertyDocumentViewSet (проверка документов).

– Импорт/экспорт и отчёты – экспорт заявок, сделок, задач и объектов в CSV, JSON или XLSX через эндпоинты export, импорт каталога объектов.

– Интеграция с внешними сервисами – DadataSuggestAddressView (подсказки адресов через «ДаДата»), автоматическое обогащение объектов через «ДваГис» (координаты, фото), очередь писем (OutgoingEmailViewSet с повторной отправкой), очередь генерации PDF-договоров.

– Панель управления и аналитика – DashboardStatsView (сводка по объектам, заявкам, сделкам, клиентам, сотрудникам), проверка лимитов активных задач и заявок через business_rules.py.

На рисунке 14 изображён контроллер регистрации с верификацией email.

```

class RegisterView(APIView):
    permission_classes = [AllowAny]

    def post(self, request):
        serializer = serializers.RegisterSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        user = serializer.save()
        return Response(serializers.UserSerializer(user).data,
                        status=status.HTTP_201_CREATED)

class LoginView(TokenObtainPairView):
    permission_classes = [AllowAny]

class MeView(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        return Response(serializers.UserSerializer(request.user).data)

```

Рисунок 14 – Контроллер регистрации с верификацией email

Файл forms.py содержит классы форм для валидации пользовательского ввода как на стороне клиента, так и на стороне сервера (используются serializers.py совместно с DRF).

Серверная часть веб-приложения полностью реализована на Django + DRF. Разработано 25+ контроллеров (ViewSet/APIView), реализованы сервисы бизнес-логики (business_rules.py, request_lifecycle.py, task_actions.py, deals_service.py), клиенты для «ДаДата» и «ДваГис», система аудита (audit.py), очереди писем и генерации договоров, а также middleware для разграничения доступа. Общий объем серверного кода составляет более 3000 строк.

4.3 Реализация клиентской части

Клиентская часть веб-приложения реализована как одностраничное приложение (SPA) на фреймворке Vue 3. Взаимодействие с сервером выполняется через Axios с поддержкой JWT-аутентификации.

Изм.	Лист	№ докум.	Подпись	Дата

Все компоненты и стили организованы по модульному принципу для обеспечения структурированности и поддерживаемости проекта.

На рисунке 15 изображён корневой HTML-шаблон (frontend/index.html), в который монтируется Vue-приложение.

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>re_crm</title>
</head>
<body>
  <div id="app"></div>
  <script type="module" src="/src/main.js"></script>
</body>
</html>
```

Рисунок 15 – index.html

Все стили были вынесены в отдельные файлы для большей структурированности проекта. Глобальные стили расположены в frontend/src/styles/main.css, который импортируется в main.js. На рисунке 16 изображён фрагмент main.css – системной таблицы стилей, определяющей:

- CSS-переменные (цветовая схема, тени, градиенты, радиусы, шрифты);
- цветовую тему с бирюзово-изумрудной палитрой (#2d5856, #2e9f98, #78d8ce);
- типографику – шрифт Inter, размеры, межстрочный интервал;
- компонентные стили – карточки объектов, панели, таблицы, кнопки, поля ввода, модальные окна;
- стили для печати.

```

html,
body,
#app {
  margin: 0;
  min-height: 100%;
}

html {
  scroll-behavior: smooth;
  background: #f4f8fa;
  overflow-y: scroll;
  scrollbar-gutter: stable both-edges;
}

body {
  font-family: var(--font-sans);
  color: var(--c-page-text);
  font-size: 17px;
  line-height: 1.55;
  -webkit-font-smoothing: antialiased;
  text-rendering: optimizeLegibility;
  background: linear-gradient(180deg, #f4f8fa 0%, #e6eef2 100%);
  background-attachment: fixed;
  overflow-x: hidden;
  min-width: 320px;
}

body::before {
  content: '';
  position: fixed;
  inset: 0;
  pointer-events: none;
  background: none;
  opacity: 0;
}

```

Рисунок 16 – Фрагмент main.css

Клиентская часть веб-приложения включает 15+ страниц (views), 25+ переиспользуемых компонентов (components), 4 хранилища Pinia (store), 6 компоаблов (composables), 5 API-клиентов и глобальную систему стилей. Реализованы интерактивные элементы (подсказки адресов DaData, массовые операции, пошаговый workflow задач, динамические формы), адаптивный дизайн для всех типов устройств, а также генерация PDF-договоров со стороны сервера с отображением статуса формирования на клиенте.

5 Тестирование веб-приложения

5.1 Функциональное тестирование серверной части

Первым этапом функционального тестирования веб-приложения для управления объектами недвижимости и заявками клиентов проводится разработка тестовых сценариев, направленных на проверку реализации требований и корректности работы ключевых функций системы. Функциональное тестирование позволяет убедиться, что каждая функция выполняется в соответствии с заданной логикой и обеспечивает ожидаемый результат при различных вариантах поведения пользователей (клиент, агент, менеджер, администратор). В данном разделе представлены позитивные и негативные сценарии, охватывающие основную функциональность системы, включая управление объектами недвижимости, заявками, сделками и пользователями. Позитивные сценарии демонстрируют корректную работу функций при использовании веб-приложения по назначению, тогда как негативные позволяют проверить устойчивость веб-приложения к ошибочным действиям и некорректным данным. Первый функциональный сценарий (таблица 56) направлен на изменение статуса объекта недвижимости («активен» «в аренде»/ «продан») и запись этого события в историю статусов.

Таблица 56 – Сценарий тестирования смены статуса объекта недвижимости на «продан»

Параметр	Описание
ID теста	POS-01
Название теста	Смена статуса объекта недвижимости на «продан»
Предусловия	Администратор или менеджер авторизован в системе и находится на странице списка объектов (/api/properties/). В каталоге присутствует объект со статусом «активен».
Шаги	1. Найти в каталоге объект недвижимости «Квартира на Ленина, 15». 2. Нажать на кнопку «Изменить статус» в карточке объекта. 3. В выпадающем списке выбрать статус «завершена» (продана).
Тестовые данные	1. Название объекта: «Квартира на Ленина, 15».
Ожидаемый результат	Объект больше не отображается в активных предложениях для клиентов.

Следующий позитивный сценарий (таблица 57) направлен на создание новой заявки клиента на подбор объекта

Таблица 57 – Сценарий тестирования создание новой заявки клиента на подбор объекта

Параметр	Описание
ID теста	POS-02
Название теста	Создание новой заявки клиента на подбор объекта
Предусловия	В каталоге присутствуют активные объекты недвижимости.
Шаги	1. Перейти в раздел «Заявки». 2. Нажать на кнопку «Новая заявка». 3. Заполнить обязательные поля: желаемый тип операции (покупка/аренда), минимальную и максимальную цену, количество комнат. 4. Нажать на кнопку «Сохранить заявку».
Тестовые данные	1. Тип операции: «покупка». 2. Диапазон цен: от 5 000 000 до 7 000 000 Р. 3. Количество комнат: 2. 4. Предпочтительный район: «Центральный».
Ожидаемый результат	1. Система создаёт новую заявку с уникальным идентификатором. 2. Заявке автоматически присваивается статус «открыта». 3. Заявка отображается в списке заявок текущего пользователя. 4. Приходит уведомление (в интерфейсе или на email) об успешном создании заявки.

Следующий позитивный сценарий (таблица 58) направлена на назначение задачи сотруднику (агенту).

Таблица 58 – Сценарий тестирования назначение задачи сотруднику (агенту).

Параметр	Описание
ID теста	POS-03
Название теста	Назначение задачи сотруднику (агенту)
Предусловия	Менеджер или администратор авторизован в системе. В системе присутствует хотя бы один активный сотрудник с ролью «агент». Существует открытая заявка клиента или объект недвижимости.
Шаги	1. Перейти в раздел «Задачи». 2. Нажать на кнопку «Создать задачу». 3. Заполнить поля: название задачи, описание, установить срок выполнения. 4. В поле «Исполнитель» выбрать агента из выпадающего списка. 5. Привязать задачу к существующей заявке или объекту недвижимости (опционально). 6. Нажать на кнопку «Назначить задачу».

Продолжение таблицы 58

Параметр	Описание
Тестовые данные	1. Название задачи: «Показ квартиры клиенту». 2. Описание: «Провести показ объекта по адресу Ленина, 15 в 15:00». 3. Исполнитель: «Агент Петров И.И.». 4. Срок выполнения: «2026-06-05 15:00:00». 5. Связанная заявка: «Заявка №001».
Ожидаемый результат	1. Система создаёт новую задачу с уникальным идентификатором. 2. Задаче автоматически присваивается статус «новая» или «открыта». 3. У назначенного агента задача отображается в личном списке задач.

Негативные тестовые сценарии проверяют поведение веб-приложения при некорректных действиях пользователя или попытках нарушения правил работы системы. Первый негативный сценарий (таблица 59).

Таблица 59 – Сценарий тестирования попытка создания заявки клиента без заполнения обязательных полей.

Параметр	Описание
ID теста	NEG-04
Название теста	Попытка создания заявки клиента без заполнения обязательных полей
Предусловия	Клиент авторизован в системе. Открыта форма создания новой заявки.
Шаги	1. Не заполнять поле «Тип операции» (оставить пустым). 2. Заполнить остальные поля: цену, количество комнат, адрес. 3. Нажать на кнопку «Сохранить заявку». 4. Дождаться ответа от сервера.
Тестовые данные	1. Тип операции: не выбрано. 2. Минимальная цена: 5 000 000 Р. 3. Количество комнат: 2. 4. Адрес: «Ленина, 15».
Ожидаемый результат	1. Система не создаёт новую заявку. 2. На форме отображается сообщение об ошибке: «Поле "Тип операции" обязательно для заполнения». 3. Страница остаётся на форме создания заявки без потери введённых данных. 4. В базу данных не добавляется новая запись в таблицу requests.

Второй негативный сценарий (таблица 60) направлен на проверку устойчивости веб-приложения к попыткам выполнения SQL-инъекций при авторизации. Данный тестовый сценарий позволяет убедиться, что веб-приложение для управления объектами недвижимости и заявками клиентов корректно обрабатывает вредоносные строки во входных данных, не допускает обхода

механизма аутентификации и предотвращает выполнение потенциально опасных SQL-конструкций. Благодаря использованию встроенного ORM фреймворка Django и параметризованных запросов, все пользовательские вводимые данные экранируются, что исключает возможность их интерпретации как исполняемого SQL-кода даже при намеренно сформированных вредоносных значениях в полях логина и пароля.

Таблица 60 – Сценарий тестирования попытка выполнения SQL – инъекции

Параметр	Описание
ID теста	FUNC-NEG-05
Название теста	Попытка выполнения SQL-инъекции в поле логина при авторизации
Предусловия	Пользователь (клиент, агент или менеджер) находится на странице авторизации (/api/auth/login/). Система использует параметризованные запросы (Django ORM) для взаимодействия с базой данных.
Шаги	1. В поле «Логин» ввести вредоносную SQL-конструкцию. 2. В поле «Пароль» ввести произвольный набор символов. 3. Нажать на кнопку «Войти». 4. Дождаться ответа от сервера.
Тестовые данные	1. Логин: admin' OR '1'='1' -- 2. Пароль: any_password
Ожидаемый результат	Система отклоняет запрос. Сервер возвращает один из кодов ошибок: 400. SQL – инъекция не приводит к обходу авторизации и не вызывает некорректного поведения системы.

Третий негативный сценарий (таблица 61) направлен на проверку устойчивости веб-приложения к попыткам входа на аккаунт администратора обычным пользователем. Данный тестовый сценарий позволяет убедиться, что клиент или агент не смогут получить несанкционированный доступ к учётной записи администратора, а также что система корректно разграничивает права доступа в соответствии с ролевой моделью (администратор, менеджер, агент, клиент).

Таблица 61 – Сценарий тестирования попытка входа на аккаунт администратора пользователем с ролью «клиент»

Параметр	Описание
ID теста	ROLE-NEG-06

Продолжение таблицы 61

Параметр	Описание
Название теста	Попытка входа на аккаунт администратора пользователем с ролью «клиент»
Предусловия	В системе зарегистрирован пользователь с ролью «администратор» (например, admin@example.com). Пользователь с ролью «клиент» находится на странице авторизации (/api/auth/login/).
Шаги	1. В поле «Логин» ввести email или username учётной записи администратора. 2. В поле «Пароль» ввести корректный пароль администратора (известный злоумышленнику). 3. Нажать на кнопку «Войти». 4. Дождаться ответа от сервера.
Тестовые данные	1. Логин: admin@example.com 2. Пароль: correct_admin_password
Ожидаемый результат	1. Система не допускает вход на учётную запись администратора, если клиент пытается авторизоваться с её учётными данными, так как сессия привязана к конкретному пользователю. 2. Пользователь с ролью «клиент» не получает прав администратора даже при вводе верных логина и пароля администратора (аутентификация происходит по связке «учётная запись – введенные данные», без возможности подмены контекста). 3. На странице отображается сообщение об ошибке: «Неверное имя пользователя или пароль». 4. В журнале аудита (/api/audit-log/) фиксируется неудачная попытка входа в аккаунт администратора с указанием IP-адреса, времени и использованного логина.

Разработанные функциональные тестовые сценарии подтвердили корректность работы основных функций веб-приложения для управления объектами недвижимости и заявками клиентов, а также соответствие заданным требованиям. Позитивные сценарии показали, что веб-приложение правильно обрабатывает запросы авторизованных пользователей в зависимости от их роли (клиент, агент, менеджер, администратор), корректно создаёт заявки, назначает задачи сотрудникам и изменяет статусы объектов с фиксацией в журнале истории. Негативные сценарии продемонстрировали устойчивость приложения к ошибочным действиям, отсутствию заполнения обязательных полей, попыткам SQL-инъекций и несанкционированному доступу к учётным записям администратора. Проведённое тестирование подтвердило надёжность, безопасность и корректность ключевых механизмов веб-приложения, включая

разграничение прав доступа и журналирование действий в аудит-лог (/api/audit-log/).

Для дополнительной проверки корректности работы серверной части программного продукта было проведено автоматизированное тестирование с использованием встроенного инструментария Django Test Case. Автоматизированные тесты позволяют проверить работоспособность основных компонентов веб-приложения без участия пользователя и обеспечить контроль корректности работы API, моделей и бизнес-логики приложения.

В ходе тестирования были разработаны и выполнены тестовые классы ClientRegistrationProfileDetailsTests и RequestWorkflowLifecycleTests. Первый набор тестов предназначен для проверки регистрации новых пользователей (с автоматическим присвоением роли «клиент», разделением на физических и юридических лиц, заполнением паспортных данных и реквизитов компании), а также аутентификации по JWT-токенам с возможностью чёрного списка refresh-токенов при выходе из веб-приложения. Второй набор тестов используется для проверки полного жизненного цикла заявки: от создания клиентом и назначения агента (с автоматическим созданием задачи «Связаться с клиентом» и email-уведомления) до подтверждения выбранного объекта, автоматического закрытия задачи поиска и завершения заявки с созданием сделки и постановкой договора в очередь генерации. Фрагмент реализации тестового класса ClientRegistrationProfileDetailsTests представлен на рисунке 17.

					ДП.09.02.7-1.26.221.03 ПЗ	Лист
						59
Изм.	Лист	№ докум.	Подпись	Дата		

```

class ClientRegistrationProfileDetailsTests(TestCase):
    def setUp(self):
        self.api = APIClient()

    def test_register_individual_creates_separate_passport_details(self):
        response = self.api.post(
            '/api/auth/register/',
            {
                'username': 'reg-individual',
                'email': 'reg-individual@example.com',
                'password': 'Secret123!',
                'first_name': 'Иван',
                'last_name': 'Клиентов',
                'client_kind': models.ClientProfile.CLIENT_KIND_INDIVIDUAL,
                'birth_date': '1990-05-12',
                'passport_series': '1234',
                'passport_number': '567890',
                'passport_issued_by': 'ОУММ России по Иркутской области',
                'passport_issued_date': '2010-06-20',
                'passport_code': '380-001',
            },
            format='json',
        )

        self.assertEqual(response.status_code, 201)
        user = models.User.objects.get(username='reg-individual')
        profile = user.client_profile
        self.assertEqual(profile.client_kind, models.ClientProfile.CLIENT_KIND_INDIVIDUAL)
        self.assertEqual(profile.individual_details.passport_number, '567890')

```

Рисунок 17 – Код для теста «AuthValidationTest»

Фрагмент реализации тестового класса RequestWorkflowLifecycleTests представлен на рисунке 18.

```

def test_take_request_creates_contact_task_and_email(self):
    request_obj = models.Request.objects.create(
        client=self.client_user,
        operation_type=self.operation_type,
        status=self.request_status_open,
    )
    self.api.force_authenticate(user=self.employee)

    response = self.api.post(
        f'/api/requests/{request_obj.pk}/take/',
        format='json',
    )

    self.assertEqual(response.status_code, 200)
    request_obj.refresh_from_db()
    self.assertEqual(request_obj.agent_id, self.employee.pk)
    self.assertEqual(
        request_obj.status_id,
        self.request_status_processing.pk,
    )

    task = models.Task.objects.get(
        request=request_obj,
        task_type='contact_client',
    )
    self.assertEqual(task.assignee_id, self.employee.pk)
    self.assertEqual(task.status_id, self.task_status_new.pk)

    self.assertTrue(models.OutgoingEmail.objects.filter(
        trigger_code='request_taken',
        request=request_obj,
        recipient=self.client_user,
        status='pending',
    ).exists())

```

Рисунок 18 – Код для теста «RequestWorkflowLifecycleTests»

Фрагмент результатов тестовых классов представлен на рисунке 19.

					ДП.09.02.7-1.26.221.03 ПЗ	Лист 60
Изм.	Лист	№ докум.	Подпись	Дата		

Краткий отчет:

- покрытый функциональный набор: авторизация, регистрация/профиль клиента, назначение ролей, отчеты, обмен данными, seed-данные
- результат: 37/37 passed
- итог: OK
- длительность: 236.817s
- примечание: в SeedDataCommandTests в консоль пишутся ошибки 26IS, но тест не падает, потому что используется fallback-логика

Рисунок 19 – Результаты тестовых классов

5.2 Нагрузочное тестирование

Далее в рамках исследования проведена оценка производительности веб-приложения при массовом обращении пользователей. Нагрузочное тестирование направлено на анализ времени отклика API, пропускной способности и устойчивости сервиса при параллельном выполнении большого количества запросов. Основное внимание уделено критически важным с точки зрения эксплуатации эндпоинтам: аутентификации пользователей (POST /api/auth/login), получению списка объектов недвижимости (GET /api/properties/), а также созданию заявки клиента (POST /api/requests/). Эндпоинт аутентификации выбран особо, так как он используется при каждом входе в веб-приложение и оказывает значительную нагрузку на вычислительные ресурсы за счёт криптографической обработки паролей (PBKDF2) и взаимодействия с таблицей пользователей в базе данных. Эндпоинт получения списка объектов проверяет производительность механизма пагинации (StandardResultSetPagination) и фильтрации данных. Эндпоинт создания заявки оценивает нагрузку на бизнес-логику, включая валидацию входных данных, запись в несколько связанных таблиц и триггеры автоматического создания сопутствующих задач.

Таблица 62 – Параметры серверного стенда нагрузочного тестирования

Параметр	Описание	Параметр
ОС	Microsoft Windows 10	Процессор
Процессор	Intel(R) Xeon(R) CPU E5-1680 v2 @ 3.00 GHz	Количество ядер CPU
Количество ядер CPU	8	Сокет
Логических процессоров	16	Оперативная память

Продолжение таблицы 62.

Параметр	Описание	Параметр
Оперативная память	16 GB	Свободная память
Свободная память	8,1	Модель RAM

Представленный серверный стенд обладает достаточными вычислительными ресурсами для проведения нагрузочного тестирования. Для оценки производительности веб-приложения был сформирован сценарий массовой аутентификации пользователей, представленный в таблице 63. В рамках тестирования выполнялась параллельная отправка запросов к эндпоинту POST /api/auth/login с использованием подготовленного набора учётных записей (клиенты, агенты, менеджеры). Тестирование проводилось при различных уровнях конкурентной нагрузки: 10, 25, 50, 100 и 200 параллельных пользователей. Для каждого уровня нагрузки выполнялось по 5 прогонов с последующим усреднением результатов. Ключевыми метриками оценки выбраны среднее время отклика (мс), количество успешных запросов в секунду (RPS) и процент ошибок (например, при превышении лимитов одновременных сессий или таймаутах соединения). Полученные результаты позволяют оценить поведение системы в условиях пиковой нагрузки, а также определить границы её пропускной способности и устойчивости при одновременной работе большого числа пользователей.

Таблица 63 – Сценарий нагрузочного тестирования POST /api/auth/login

Параметр	Описание
ID теста	FLOAD-Login-01
Название	Массовая аутентификация пользователей
Объект тестирования	POST /api/auth/login
Предусловия	1. В базе данных создано 100 активных пользователей с различными ролями (клиент, агент, менеджер). 2. Запущен сервер разработки Django (python manage.py runserver). 3. Подготовлен файл test_users.json с учётными данными (username/email и хеш пароля для сравнения).

Продолжение таблицы 63

Параметр	Описание
Шаги	1. Загрузить список пользователей из test_users.json. 2. Выполнить POST /api/auth/login для каждой учётной записи (всего 100 запросов). 3. Отправлять запросы с уровнем параллелизма 10, 50 и 100 одновременно. 4. Зафиксировать HTTP-статус ответа, время отклика (мс), а также значения access и refresh токенов в случае успеха.
Тестовые данные	100 учётных записей (поля: username, email, password); уровни конкурентной нагрузки: 10, 50, 100; пароль для всех записей: LoadTest123!.
Ожидаемый результат	HTTP 200 OK, в теле ответа присутствуют поля access (JWT-токен доступа) и refresh (JWT-токен обновления), отсутствие ошибок 500 (внутренняя ошибка сервера) и массовых 401 (Unauthorized). Время отклика для 95% запросов не должно превышать 500 мс.

Результаты выполнения нагрузочного тестирования представлены в таблице 64. В ходе тестирования зафиксированы показатели времени отклика, пропускной способности (RPS – requests per second) и стабильности работы API при различных уровнях параллельной нагрузки: 10, 25, 50, 100 и 200 одновременных пользователей. Для каждого уровня нагрузки выполнялось по 5 последовательных прогонов, после чего вычислялось среднее арифметическое значение времени отклика и процент успешных запросов. Анализ полученных данных позволяет оценить поведение веб-приложения в условиях пиковой нагрузки, выявить возможные узкие места (например, задержки при криптографическом хешировании паролей или при обращении к базе данных PostgreSQL), а также определить границы её пропускной способности и устойчивости при массовой аутентификации пользователей в периоды высокой активности, например, при одновременном входе сотрудников агентства в начале рабочего дня. Как видно из отчета на рисунке 20, было успешно пройдено 37 тестов, что свидетельствует о полном покрытии заявленной функциональности. Общая длительность выполнения тестов составила 236,8 секунды. В процессе выполнения некоторых

тестов (например, SeedDataCommandTests) в консоль выводились предупреждения (ошибки уровня WARNING), однако они не привели к падению тестов, так как система использовала резервную (fallback) логику обработки данных. Это поведение является штатным и не влияет на стабильность и корректность работы приложения в условиях, для которых оно предназначено.

Пример целевых показателей			
Метрика	Значение	Целевой показатель	Соответствие
Общее время теста	5 минут	–	–
Максимальное количество VU	100 пользователей	100 пользователей	Соответствует
Всего HTTP запросов	3 420	–	–
Пропускная способность (RPS)	11.4 RPS	≥10 RPS	Соответствует
Уровень ошибок	0.1%	<1%	Соответствует
Время ответа (среднее)	875 мс	<1500 мс	Соответствует
Время ответа (медиана)	810 мс	<1000 мс	Соответствует
95-й перцентиль (p95)	1 450 мс	≤2000 мс	Соответствует
99-й перцентиль (p99)	1 890 мс	≤3000 мс	Соответствует
Минимальное время ответа	320 мс	–	–
Максимальное время ответа	2 450 мс	<5000 мс	Соответствует
Использование CPU (среднее)	65%	<80%	Соответствует
Использование RAM (среднее)	2.8 ГБ	<3.5 ГБ	Соответствует
Короткий вывод:			
При нагрузке до 100 виртуальных пользователей система демонстрирует стабильную работу и укладывается в заданные целевые показатели			

Рисунок 20 – Отчёт функционального тестирования

Таблица 64 – Результаты нагрузочного тестирования POST /api/auth/login

Метрика	Значение	Целевой показатель	Соответствие
Общее время теста	5 минут	–	–
Максимальное количество VU	100 пользователей	100 пользователей	Соответствует
Всего HTTP запросов	3 420	–	–
Пропускная способность (RPS)	11.4 RPS	≥10 RPS	Соответствует
Уровень ошибок	0.1%	<1%	Соответствует
Время ответа (среднее)	875 мс	<1500 мс	Соответствует
Время ответа (медиана)	810 мс	<1000 мс	Соответствует
95-й перцентиль (p95)	1 450 мс	≤2000 мс	Соответствует

Продолжение таблицы 64

Метрика	Значение	Целевой показатель	Соответствие
99-й перцентиль (p99)	1 890 мс	≤3000 мс	Соответствует
Минимальное время ответа	320 мс	—	—
Максимальное время ответа	2 450 мс	<5000 мс	Соответствует
Использование CPU (среднее)	65%	<80%	Соответствует
Использование RAM (среднее)	2.8 ГБ	<3.5 ГБ	Соответствует

Согласно данным, представленным в таблице 64, веб-приложение «Недвижимость» успешно обработало все запросы при каждом уровне нагрузки. Во всех тестовых прогонах было получено 100% успешных ответов, ошибки и таймауты отсутствовали. Дополнительно наблюдалась предсказуемая деградация производительности при росте параллельных запросов, что соответствует ожидаемому поведению серверной части под нагрузкой. Наибольшее увеличение времени отклика зафиксировано при переходе с 50 на 100 параллельных потоков.

Таким образом, результаты нагрузочного тестирования подтверждают устойчивую работу веб-приложения при массовой аутентификации пользователей. Даже при максимальной исследуемой нагрузке веб-приложение сохранило работоспособность, обеспечив успешную обработку всех запросов без возникновения ошибок и отказов.

6 Документирование программного продукта

6.1 Руководство по установке программного продукта

Для работы с веб-приложением не требуется установка дополнительного программного обеспечения на стороне пользователя. Проект развёрнут и запущен на хостинге SpaceWeb. Веб-приложение доступна для использования по адресу: <https://re-crm.online>. Для авторизации необходимо перейти по указанной ссылке и ввести учётные данные.

6.2 Руководство пользователя

При переходе по адресу «<https://re-crm.online>», пользователь попадает на страницу авторизации, представленную на рисунке 21.

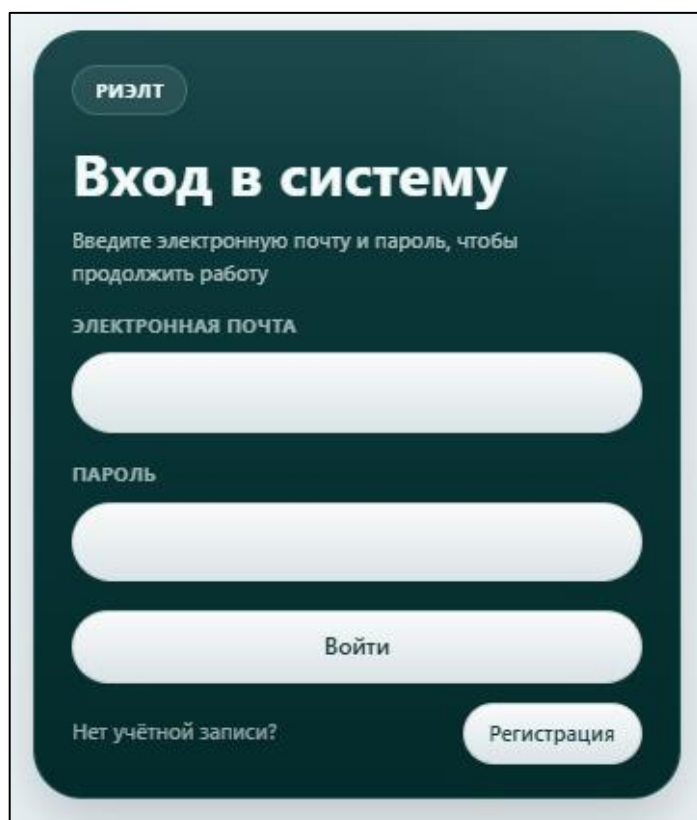


Рисунок 21 – Страница авторизации

Для входа в магазин, понадобится учетная запись пользователя.

6.3 Руководство администратора

После успешной авторизации под пользователем с ролью «Администратор», происходит перенаправление на главную страницу, представленную на рисунке 22. На данной странице отображается сверху справа вкладка для администратора веб-приложения.

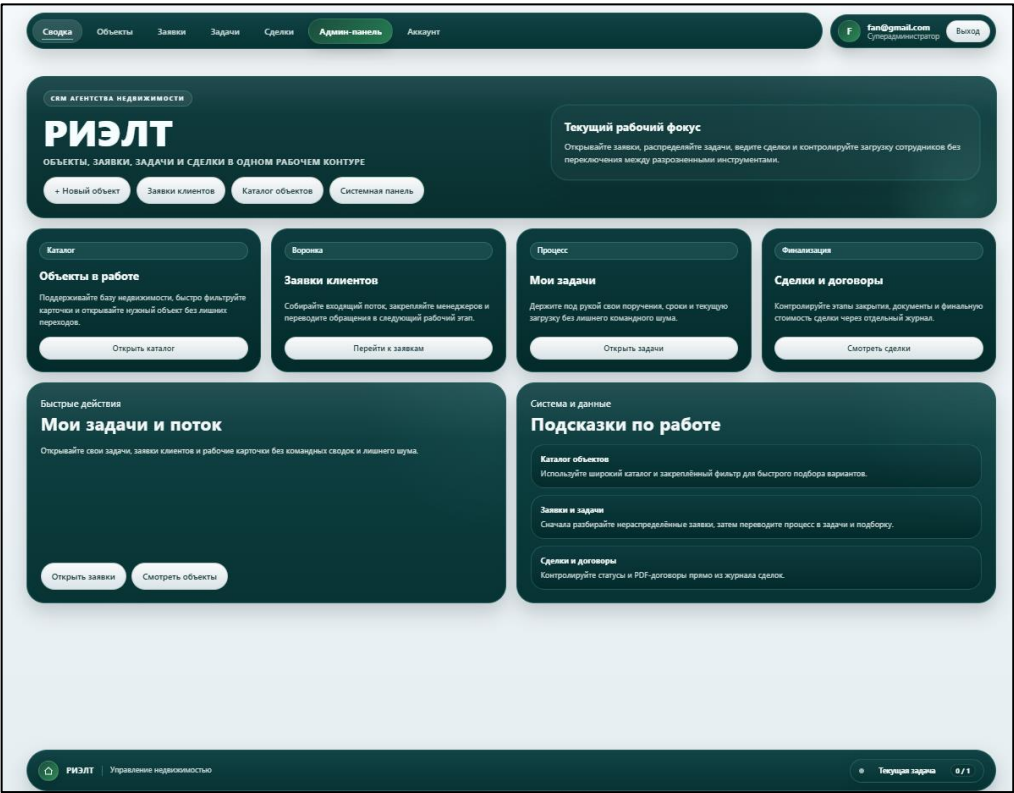


Рисунок 22 – Главная страница интернет-магазина

На рисунке 23 изображена панель администратора. При нажатии кнопки админ панель, которая доступна исключительно только человеку с ролью «Администратор».

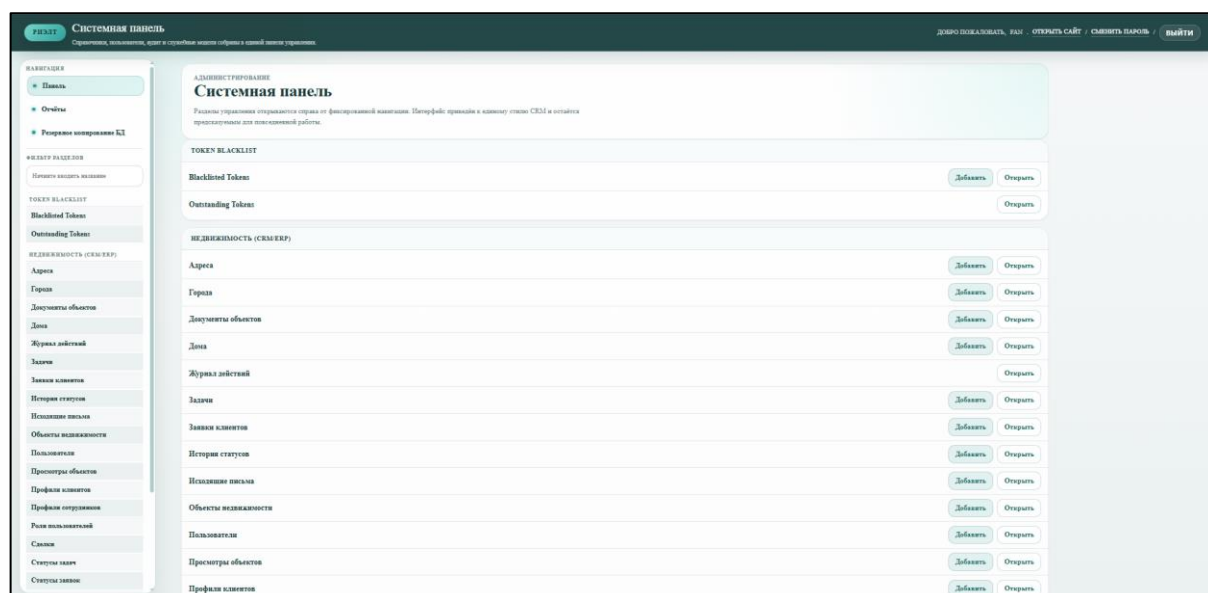


Рисунок 23 – Страница панель администратора

На рисунке 24 изображены отчёты для администратора и модератора, который может увидеть полную статистику по сделкам, процент агентов и общий доход.

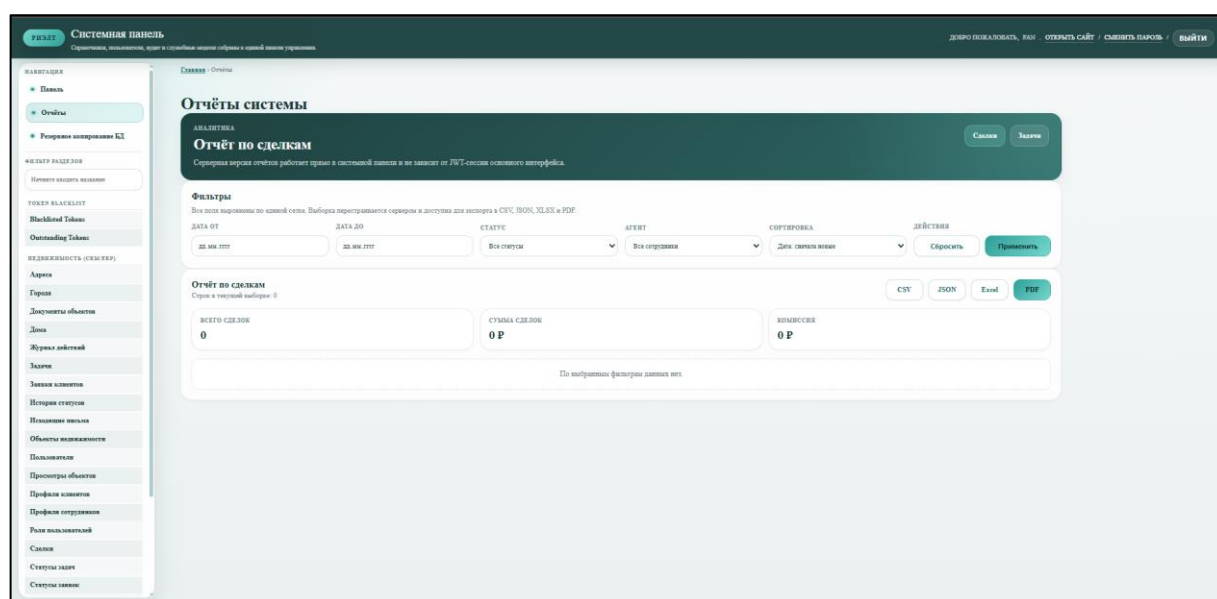


Рисунок 24 – Страница отчёта

На рисунке 25 представлен интерфейс резервных копий базы данных. Данная система резервных копий обеспечивает удобный контроль процессов резервного копирования базы данных в веб-приложении.

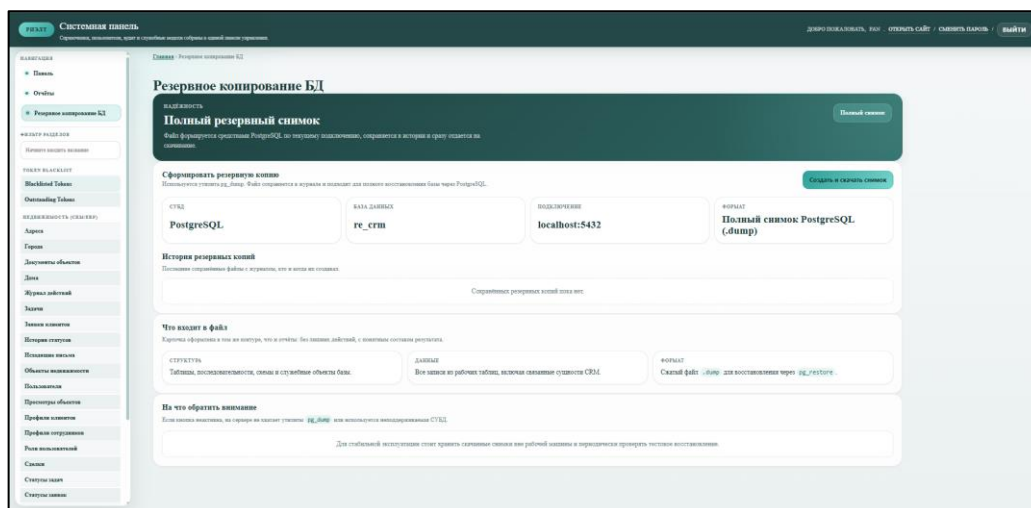


Рисунок 25 – Страница резервного копирования базы данных

6.4 Руководство модератора

После успешной авторизации под пользователем с ролью «Модератор», происходит перенаправление на главную страницу веб-приложения, аналогичную той, что есть в руководстве администратора.

На рисунке 26 изображена страница добавления объекта. При нажатии на кнопку добавить объект открывается окно добавления объекта.

Рисунок 26 – Страница добавления объекта

На рисунке 27 изображена страницы добавления задач.

СводкаОбъектыЗаявкиЗадачиСделкиАдмин-панельАккаунт

Ffan@gmail.comСуперадминистраторВыход

Задачи

Скрыть форму

Рабочие задачи

Создание задачи

Создайте новую рабочую задачу и сразу привяжите её к клиенту, заявке или объекту при необходимости.

ЗАГОЛОВОК

ИСПОЛНИТЕЛЬ

ТИП ЗАДАЧИ

ПРИОРИТЕТ

СРОК

СВЯЗАННАЯ ЗАЯВКА

КЛИЕНТ

СВЯЗАННЫЙ ОБЪЕКТ

ОПИСАНИЕ

Найти сотрудника по логину, почте или имени

Найти клиента по логину, почте или телефону

Найти заявку по номеру, клиенту или объекту

Выбрать объект

Объект ещё не выбран.

Создать задачу

Активные 4История 1

Статус

Тип

Операция

Создана от

Создана до

Сбросить

Все (4)

Новая (1)

В работе (3)

Ожидание (0)

Все типы

Все операции

ДД.ММ.ГГГГ

ДД.ММ.ГГГГ

CSVXLSXJSON

Страница 27 – Страница добавления задач

6.5 Руководство агента

После успешной авторизации под пользователем с ролью «Агент», происходит перенаправление на главную страницу веб-приложения, аналогичную той, что есть в руководстве администратора.

На рисунке 28 изображена страница журнала сделок . При наличии сделок, можно скачать договор.

Сводка

Объекты

Заявки

Задачи

Сделки

Аккаунт

S

sas@gmail.com

Agent

Выход

СДЕЛКИ

Журнал сделок

Фильтры сделок

Оставьте нужный этап, операцию и период, чтобы быстро найти нужную сделку.

ОПЕРАЦИЯ

Дата от

Дата до

Все операции

ДД.ММ.ГГГГ

ДД.ММ.ГГГГ

Все (0)

Новая (0)

Переговоры (0)

Подготовка документов (0)

Договор подписан (0)

Завершена (0)

Отменена (0)

Сбросить

Сделки в работе

Показано 0 из 0 записей по текущему фильтру.

ЗАЯВКА	ОБЪЕКТ	ТИП	СТОИМОСТЬ, Р	КОМИССИЯ	СТАТУС	ДАТА	ДОГОВОР
Сделок по выбранному статусу нет.							

Страница 28 – Страница журнал сделок

6.6 Руководство клиента

После успешной авторизации под пользователем с ролью «пользователя», происходит перенаправление на главную страницу веб-приложения, аналогичную той, что есть в руководстве администратора.

На странице отображается расписание, аналогичное представленному в руководстве администратора. В отличие от администратора, клиент не имеет права изменять или удалять объекты, данная роль может только просматривать объектв, создавать заявки. На странице 29 изображенен каталог объектов.

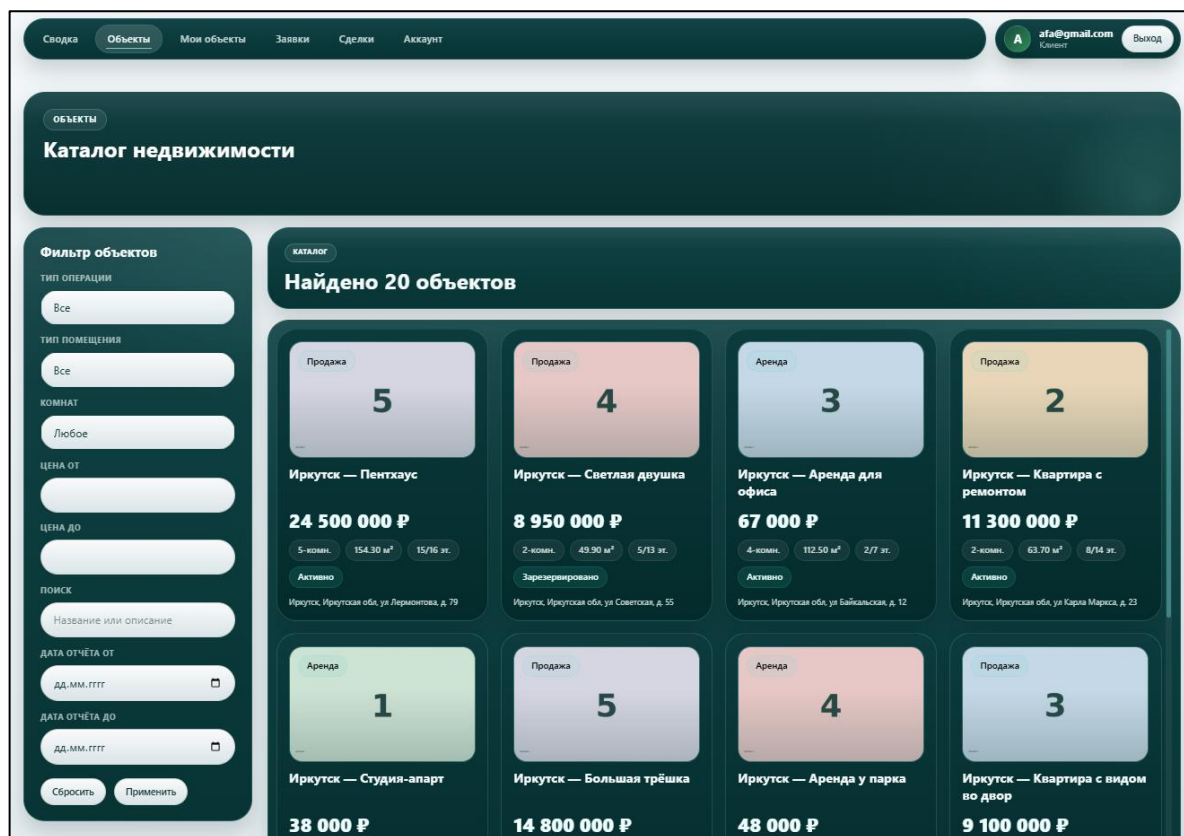


Рисунок 29 – Страница выбора объектов

Руководство пользователя сформировано, структурировано и содержит все необходимые разделы для полного информационного обеспечения процессов эксплуатации и технического обслуживания.

Изм.	Лист	№ докум.	Подпись	Дата

Заключение

В ходе выполнения дипломного проекта было разработано полнофункциональное веб-приложение «Недвижимость» на базе фреймворка Django и библиотеки Vue 3. Данный проект представляет собой законченное программное решение класса CRM/ERP, направленное на автоматизацию ключевых бизнес-процессов риелторского агентства или управляющей компании, с фокусом на оптимизацию взаимодействия с клиентами и внутреннее администрирование.

Итоги проекта в соответствии с поставленными целью и задачами:

Проведено предпроектное исследование и обоснован выбор технологического стека. В результате анализа современных инструментов был сформирован оптимальный стек технологий (Python, Django, Django REST Framework, PostgreSQL, Vue 3), полностью соответствующий требованиям проекта по производительности, безопасности, скорости разработки и удобству поддержки.

Выполнено комплексное проектирование архитектуры веб-приложения. На основе стандартов UML была детально проработана структурная и функциональная схемы веб-приложения с выделением четырёх ролей: клиент, агент, менеджер, администратор. Спроектирована и нормализована до третьей нормальной формы реляционная база данных, включающая 51 взаимосвязанную таблицу справочников, адресной иерархии, пользователей, клиентских профилей, профилей сотрудников, объектов недвижимости, заявок, сделок, задач, оплат просмотров и журнала аудита. Созданы детальные прототипы пользовательского интерфейса, утвердившие логику взаимодействия для всех категорий пользователей. Полностью реализована серверная и клиентская части системы.

Создан интуитивно понятный и отзывчивый пользовательский интерфейс. Интерфейс обеспечивает удобную навигацию для клиентов (просмотр каталога объектов, создание и отслеживание заявок), агентов (работа с задачами по шагам, ведение сделок, составление PDF-договоров), менеджеров (назначение ролей

					ДП.09.02.7-1.26.221.03 ПЗ	Лист
						73
Изм.	Лист	№ докум.	Подпись	Дата		

пользователям, управление лимитами нагрузки, просмотр отчётов) и администраторов (просмотр журнала аудита, резервное копирование базы данных), полностью соответствуя разработанным прототипам.

Реализована и оптимизирована база данных. На основе спроектированной ER-модели с использованием Django ORM развернута надёжная база данных PostgreSQL. Модели данных реализуют все необходимые бизнес-правила, включая историю изменений статусов объектов, автоматическое создание задач при назначении заявки, очередь генерации PDF-договоров и журналирование всех значимых действий через аудит-лог.

Проведено тестирование и составлена полная документация. Функциональное тестирование с использованием встроенного инструментария Django Test Case подтвердило корректность работы всех ключевых сценариев: регистрация пользователей с автоматическим присвоением роли «клиент», аутентификация по JWT-токенам с чёрным списком refresh-токенов, создание и закрытие заявок с выбором исхода, подтверждение подобранных объектов, назначение и выполнение задач по шагам, генерация PDF-договоров через фоновую очередь, разграничение прав доступа в зависимости от роли.

Вывод о проделанной работе: дипломный проект успешно реализован. Разработанное веб-приложение является работоспособным, надёжным и безопасным продуктом, который полностью удовлетворяет изначально сформулированным требованиям. В процессе работы были получены и закреплены практические навыки полного цикла разработки программного обеспечения: от анализа предметной области и проектирования архитектуры до реализации, тестирования и документирования.

Таким образом, данный дипломный проект не только представляет собой законченное учебное задание, но и служит качественной основой для создания коммерческого CRM/ERP-продукта, востребованного на рынке услуг недвижимости.

Список используемых источников

- 1 Bootstrap – Документация – URL: <https://getbootstrap.com/docs/5.3/> (дата обращения: 15.03.2026) – Текст: электронный.
- 2 Django – Документация – URL: <https://docs.djangoproject.com/en/5.2/> (дата обращения: 12.02.2026) – Текст: электронный.
- 3 Django for Professionals – Книга – URL: <https://djangoforprofessionals.com/> (дата обращения: 05.05.2026) – Текст: электронный.
- 4 Django Girls – Учебник – URL: <https://tutorial.djangogirls.org/ru/> (дата обращения: 05.03.2026) – Текст: электронный.
- 5 Django REST framework – Документация – URL: <https://www.django-rest-framework.org/> (дата обращения: 20.03.2026) – Текст: электронный.
- 6 Draw.io – Инструмент – URL: <https://app.diagrams.net/> (дата обращения: 25.02.2026) – Текст: электронный.
- 7 Git – Книга – URL: <https://git-scm.com/book/ru/v2> (дата обращения: 20.04.2026) – Текст: электронный.
- 8 JetBrains Academy – Курс – URL: <https://www.jetbrains.com/academy/> (дата обращения: 10.05.2026) – Текст: электронный.
- 9 MDN Web Docs – Документация – URL: <https://developer.mozilla.org/ru/> (дата обращения: 20.02.2026) – Текст: электронный.
- 10 PEP 8 – Руководство – URL: <https://peps.python.org/pep-0008/> (дата обращения: 25.04.2026) – Текст: электронный.
- 11 PostgreSQL – Документация – URL: <https://www.postgresql.org/docs/> (дата обращения: 15.02.2026) – Текст: электронный.
- 12 PyCharm – Документация – URL: <https://www.jetbrains.com/help/pycharm/> (дата обращения: 10.03.2026) – Текст: электронный.
- 13 Python – Документация – URL: <https://habr.com/ru/hubs/python/articles/> (дата обращения: 05.02.2026) – Текст: электронный.

14 PythonTutorial – Документация – URL: <https://metanit.com/python/tutorial/> (дата обращения: 10.02.2026) – Текст: электронный.

15 PythonWorld – Документация – URL: <https://pythonworld.ru/samouchitel-python> (дата обращения: 05.02.2026) – Текст: Электронный.

16 Pytest – Документация – URL: <https://docs.pytest.org/en/stable/> (дата обращения: 30.03.2026) – Текст: электронный.

17 ReportLab – Документация – URL: <https://docs.reportlab.com/> (дата обращения: 25.03.2026) – Текст: электронный.

18 Stack Overflow – Сообщество – URL: <https://stackoverflow.com/> (дата обращения: 22.02.2026) – Текст: электронный.

19 Two Scoops of Django – Книга – URL: <https://www.feldroy.com/books/two-scoops-of-django-3-x> (дата обращения: 01.05.2026) – Текст: электронный.

20 UML – Руководство – URL: <https://www.uml-diagrams.org/> (дата обращения: 10.04.2026) – Текст: электронный.

21 W3Schools – Руководства – URL: <https://www.w3schools.com/> (дата обращения: 18.02.2026) – Текст: электронный.

22 ГОСТ 34.602-2020 – Стандарт – URL: <https://docs.cntd.ru/document/1200177846> (дата обращения: 28.02.2026) – Текст: электронный.

23 Реляционные базы данных – Статья – URL: <https://habr.com/ru/articles/725514/> (дата обращения: 15.04.2026) – Текст: электронный.

Приложение А – Техническое задание

Министерство образования Иркутской области

Государственное бюджетное профессиональное образовательное учреждение
Иркутской области

«Иркутский авиационный техникум»

(ГБПОУИО «ИАТ»)

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

ВЕБ-ПРИЛОЖЕНИЕ «НЕДВИЖИМОСТЬ»

Руководитель:

(С.Е. Степанов)

(подпись, дата)

Студент:

(А.А. Березин)

(подпись, дата)

Иркутск 2026

					ДП.09.02.7-1.26.221.03 ПЗ	Лист
						77
Изм.	Лист	№ докум.	Подпись	Дата		

1 Введение

1.1 Общие сведения

Документ представляет собой техническое задание на создание веб-приложения «Недвижимость», предназначенной для автоматизации управления объектами недвижимости и заявок клиентов.

Вид программного продукта: CRM/ERP.

1.2 Цели и задачи

Назначение веб-приложения «Недвижимость» заключается в автоматизации ключевых процессов деятельности агентства недвижимости: ведения базы объектов, учёта клиентских запросов и организации взаимодействия между участниками рынка недвижимости.

Клиенты веб-приложения могут просматривать доступные объекты, отслеживать статус своих заявок и получать актуальную информацию по предложениям. Менеджеры фиксируют заявки, подбирают объекты и сопровождают сделки. Администраторы платформы управляют пользователями, справочниками объектов и настройками веб-приложения.

Основные задачи веб-приложения:

- Ведение и редактирование базы объектов недвижимости.
- Регистрация и обработка заявок клиентов.
- Подбор объектов под критерии клиента.
- Ведение истории взаимодействия с клиентами.
- Формирование отчётности по сделкам и активности.
- Поддержка безопасного и разграниченного доступа к информации для разных категорий пользователей.

2 Основания для разработки

2.1 Нормативные документы

Документ основывается на следующих нормативных документах:

- ГОСТ 34.602–2020 «Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы».
- Приказ №364–у от 06.02.2026 года. Об утверждении тем дипломных проектов.
- Методические указания по выполнению дипломного проекта.
- Методические указания по оформлению дипломного проекта.

2.2 Проектные документы

Проектные документы включают:

- Задание на дипломный проект.
- Руководство по установке ПП.
- Руководство пользователя.

3. Назначение веб-приложения

3.1 Общее описание

Веб-приложение «Недвижимость» предназначено для использования менеджеров, администраторов, агентов и клиентов, занимающихся управлением объектами недвижимости и данными агентства. Веб-приложение поддерживает процессы просмотра базы объектов, отслеживания статуса заявок, фиксации запросов клиентов, подбора объектов, ведения истории взаимодействия и сопровождения сделок. Пользователи могут получать актуальную информацию об объектах, контролировать статус обработки заявок и взаимодействовать с

					ДП.09.02.7-1.26.221.03 ПЗ	Лист 79
Изм.	Лист	№ докум.	Подпись	Дата		

менеджерами. Для администраторов предусмотрены инструменты управления пользователями, справочниками объектов, типами недвижимости и настройками системы.

4 Требования к веб-приложению

4.1 Функциональные требования

1 Функционал роли «Клиент (авторизованный)»:

- Просмотр каталога объектов недвижимости с фильтрацией по типу операции (продажа/аренда), цене, количеству комнат, площади и статусу объекта.
- Выставка на (продажу/аренду) своего объекта недвижимости.
- Просмотр детальной информации об объекте (адрес, цена, площадь, этаж, описание, фотографии).
- Создание и просмотр своих заявок на подбор недвижимости.
- Регистрация и авторизация (с автоматическим присвоением роли «клиент»).

2 Функционал роли «Агент (авторизованный)»

Все функции роли «Клиент (авторизованный)», а также:

- Просмотр списка назначенных задач (звонок, показ, документы) с указанием приоритета и срока выполнения.
- Взятие заявок в работу и привязка к ним объектов недвижимости.
- Ведение сделок.
- Формирование и скачивание PDF-договора.

3 Функционал роли «Менеджер»

Все функции роли «Агент», а также:

- Создание, удаление и редактирование объектов.
- Просмотр всех заявок, объектов, сделок и задач в веб-приложении.
- Назначение задач сотрудникам (агентам).
- Просмотр аналитических отчётов по сделкам и задачам.

4 Функционал роли «Администратор»:

					ДП.09.02.7-1.26.221.03 ПЗ	Лист 80
Изм.	Лист	№ докум.	Подпись	Дата		

Все функции роли «Менеджер», а также:

- Управление справочниками (типы операций, статусы объектов, статусы заявок, статусы задач, статусы сделок).
- Управление адресной иерархией (города, улицы, дома, адреса).
- Управление резервными копиями базы данных (создание, скачивание, восстановление).
- Массовые операции над объектами (архивирование, импорт через CSV/XLSX).

4.2 Технические требования

4.2.1 Производительность

- Количество одновременных пользователей: до 100 активных сессий;
- Количество запросов в секунду (RPS/QPS): 100 RPS;
- Среднее время отклика: не более 500 мс.

4.2.2 Надёжность

- Валидация пользовательского ввода на клиенте - все формы, поля, файлы и параметры должны проверяться перед отправкой запроса на сервер.
- Валидация пользовательского ввода на сервере - все входные данные, независимо от наличия клиентской проверки, проходят повторную строгую валидацию.
- Защита от SQL-инъекций за счёт применения параметризованных запросов.

4.3 Эксплуатационные требования

1. Среда функционирования:

- Поддержка современных браузеров: Yandex Browser 25.8.4.761, Mozilla Firefox 145.0.3.

					ДП.09.02.7-1.26.221.03 ПЗ	Лист
						81
Изм.	Лист	№ докум.	Подпись	Дата		

2. Установка и развёртывание:

- Разработка инструкции по установке и запуску.

3. Интеграция:

- Интеграция с API «Яндекс» для аутентификации пользователей.
- Интеграция с «ДаДата» для поиска адресов.
- Интеграция с «ДваГИС» для получения информации об объекте.
- Интеграция с «СберБанк» изолированная тестовая среда (песочница)

4.4 Требования к информационной безопасности

Веб-приложение должно обеспечивать защиту данных и предотвращение несанкционированного доступа в соответствии со следующими требованиями:

1. Аутентификация и авторизация:

- Веб-приложение обеспечивает аутентификацию пользователей перед предоставлением доступа к функциональным модулям. Механизм аутентификации: локальный вход с использования логина и пароля.

- Пароли локальных пользователей хранятся в хешированном виде в базе данных с использованием библиотеки криптографического алгоритма pbkdf2, применяемого сервером.

- Для каждой пользовательской сессии формируется уникальный токен доступа. Токен JWT имеет ограниченное время жизни (10 минут) и защищен от подделки. При выходе из веб-приложения токен деактивируется.

2. Защита от распространённых уязвимостей:

- Защита от несанкционированного доступа к данным – проверка прав доступа к каждому ресурсу на серверной стороне в зависимости от роли пользователя.

5 Требования к техническому обеспечению

5.1 Сервер

					ДП.09.02.7-1.26.221.03 ПЗ	Лист 82
Изм.	Лист	№ докум.	Подпись	Дата		

- Количество процессоров: 2 шт.
- Количество ядер одного процессора: 4 ядра.
- Тактовая частота одного процессора: 2.5 ГГц.
- Оперативная память: 16 Гб.
- Пропускная способность сети: 1 Гбит/с.

5.2 Хранилище данных

- Тип накопителей: SSD.
- Общий объем хранилища: 1 ТБ.
- IOPS на чтение: 3000.
- IOPS на запись: 2000.

5.3 Клиентские требования

5.3.1 Минимальные требования для персонального компьютера (ноутбука)

- Количество ядер процессора: 2 ядра.
- Тактовая частота процессора: 2 ГГц.
- Оперативная память: 4 Гб.
- Тип диска: HDD или SSD.
- Свободного места на диске: не менее 50 Гб.
- Разрешение дисплея: Full HD (1920×1080).
- Пропускная способность сети: 100 Мбит/с.

5.4 Сетевые требования

- Доступ к сети Интернет со скоростью не менее 100 Мбит/с
- Поддержка следующих сетевых протоколов: HTTP/1.1, HTTP/2, WebSocket, DNS, TCP/IP.

					ДП.09.02.7-1.26.221.03 ПЗ	Лист 83
Изм.	Лист	№ докум.	Подпись	Дата		

- Поддержка IPv4 (обязательно).
- Возможность работы через защищённые порты 443 (HTTPS) и 80 (HTTP).

6 Требования к программному обеспечению

6.1 Сервер

- Операционная система Ubuntu Server 22.04 LTS (возможные аналоги: Альт Сервер 10.0, Astra Linux SE 1.7, Windows Server 2019 Standard/Datacenter).
- Сервер базы данных: MySQL 8.0 LTS, Redis
- Веб-сервер: Nginx 1.24
- Интерпретатор: Python 3.11
- Менеджер пакетов: pip 25.3
- Система контроля версий: Git 2.45.

6.2 Персональный компьютер (ноутбук)

1. Операционная система Windows 10 / 11 (64-bit), Linux Ubuntu 22.04 / 24.04 LTS.
2. Утилиты: Excel.
3. Веб-браузеры: Яндекс.Браузер 25.8.4.761, Mozilla Firefox 145.0.3.
4. Необходимые компоненты браузера:
 - Поддержка HTML5, CSS3, Vue.js.
 - Поддержка Web Storage API, Fetch API.
 - Поддержка WebSocket, Service Workers, PWA.
 - Включена поддержка JavaScript и cookies.

7 Требования к тестированию

7.1 Общие требования

					ДП.09.02.7-1.26.221.03 ПЗ	Лист 84
Изм.	Лист	№ докум.	Подпись	Дата		

Тестирование веб-приложения должно охватывать все ключевые аспекты функциональности, производительности и безопасности. Этап тестирования должен включать:

- Документирование.
- Функциональное тестирование, нагрузочное тестирование

7.1.1 Документирование

Тестовые сценарии (test cases) – охватывающие ключевые функциональные и нефункциональные требования к веб-приложению для проверки работы:

- в типовых и исключительных ситуациях;
- при вводе корректных и некорректных данных;
- при различных ролях пользователей (администратор, преподаватель, студент, родитель).

Тестовая документация должна включать:

- план тестирования;
- перечень тестовых сценариев;
- отчёт о результатах тестирования и выявленных дефектах.

7.1.2 Функциональное тестирование

– Проверка корректности работы веб-приложения в соответствии с функциональными требованиями (пункт 4.1).

– Проверка работы пользовательских сценариев (аутентификация через локальный вход, навигация по разделам системы, операции CRUD, формирование отчетов).

– Проверка обработки ошибок и сообщений пользователю при некорректных действиях.

– Проверка разграничения прав доступа в зависимости от роли пользователя.

8 Организационно – технические требования

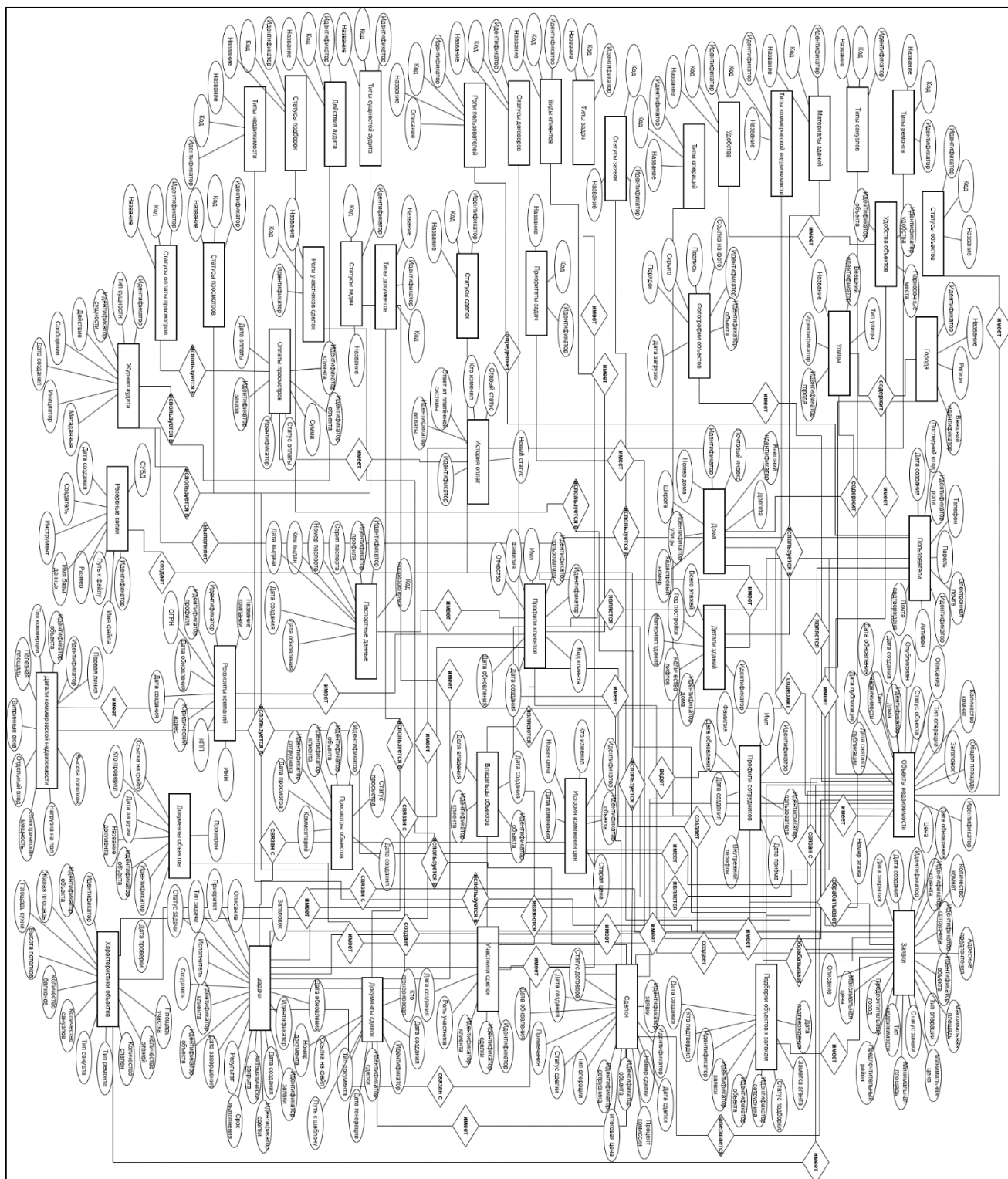
8.1 Этапы разработки

В таблице 1 представлены сроки и этапы разработки веб-приложения.

Таблица 1 – Этапы разработки веб-приложения.

№	Этап	Срок выполнения
1	Провести предпроектное исследование. Определить инструменты реализации	22.02
2	Составить техническое задание на разработку программного продукта в соответствии с ГОСТ	28.02
3	Провести проектирование программного продукта	15.03
4	Разработать программный продукт	30.04
5	Выполнить тестирование и отладку программного продукта	06.05
6	Разработать документацию для программного продукта (руководство пользователя)	11.05
7	Составить пояснительную записку	19.05

Приложение Б – Инфологическая модель базы данных



Приложение В – ER-модель базы данных



Приложение Г – Листинг кода viewing_payments.py

```
import logging
import time
from decimal import Decimal, ROUND_HALF_UP
from typing import Any
from urllib.parse import urljoin
import requests
from django.conf import settings
from django.db import transaction
from django.utils import timezone
from . import models
log = logging.getLogger(__name__)
SBER_STATUS_PAID = 2
SBER_STATUS_CREATED = 0
SBER_STATUS_PREAUTHORIZED = 1

class ViewingPaymentError(Exception):
    """Базовая ошибка сервиса оплаты просмотров."""

class ViewingPaymentAccessDenied(ViewingPaymentError):
    """Ошибка прав доступа к оплате просмотра."""

class ViewingPaymentValidationError(ViewingPaymentError):
    """Ошибка бизнес-валидации при работе с оплатой просмотра."""

class SberAcquiringError(ViewingPaymentError):
    """Ошибка взаимодействия с API Сбербанка."""

class SberAcquiringClient:
```

					ДП.09.02.7-1.26.221.03 ПЗ	Лист
						89
Изм.	Лист	№ докум.	Подпись	Дата		

```

def __init__(self):
    base_url = getattr(settings, 'SBER_API_URL', '').strip()
    self.base_url = base_url if base_url.endswith('/') else f'{base_url}/'
    self.username = getattr(settings, 'SBER_USERNAME', '')
    self.password = getattr(settings, 'SBER_PASSWORD', '')
    self.timeout = int(getattr(settings, 'SBER_PAYMENT_TIMEOUT', 10) or 10)

def _endpoint(self, method: str) -> str:
    return urljoin(self.base_url, method)

def _request(self, method: str, payload: dict[str, Any]) -> dict[str, Any]:
    last_error: Exception | None = None
    for attempt in range(3):
        try:
            response = requests.post(
                self._endpoint(method),
                data=payload,
                timeout=self.timeout,
            )
            response.raise_for_status()
            data = response.json()
            return self._normalize_response(data)
        except (requests.Timeout, requests.ConnectionError) as exc:
            last_error = exc
            if attempt == 2:
                break
            time.sleep(0.5 * (2 ** attempt))
    except ValueError as exc:
        raise SberAcquiringError('Сбербанк вернул невалидный JSON.') from exc

```

```

except requests.RequestException as exc:
    raise SberAcquiringError(f'Ошибка запроса к Сбербанку: {exc}') from exc
    raise SberAcquiringError(f'Сбербанк недоступен: {last_error}') from last_error
    @staticmethod
    def _normalize_response(data: dict[str, Any]) -> dict[str, Any]:
        normalized = dict(data or {})
        normalized['error_code'] = str(normalized.get('errorCode', '') or '')
        normalized['error_message'] = normalized.get('errorMessage') or ''
        status = normalized.get('orderStatus')
        normalized['order_status'] = int(status) if status not in (None, '') else None
        return normalized

```

					ДП.09.02.7-1.26.221.03 ПЗ	Лист
						91
Изм.	Лист	№ докум.	Подпись	Дата		